European Journal of Operational Research 298 (2022) 182–201

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Production, Manufacturing, Transportation and Logistics

Order batching and batch sequencing in an AMR-assisted picker-to-parts system

Ivan Žulj [a,*], Hagen Salewski [b], Dominik Goeke [c], Michael Schneider [d]

[a] *Department of Procurement and Production, University of Hohenheim, Germany*
[b] *Chair of Business Information and Operations Research, University of Kaiserslautern, Germany*
[c] *Lufthansa Systems GmbH & Co. KG, Germany*
[d] *Deutsche Post Chair – Optimization of Distribution Networks, RWTH Aachen University, Germany*

ARTICLE INFO

Article history:
Received 19 July 2020
Accepted 19 May 2021
Available online 27 May 2021

Keywords:
Heuristics
Autonomous mobile robots
Warehouse management
Order batching
Zone picking

ABSTRACT

Efficient order fulfillment processes in warehouses are a key success factor in times of increasing global retail sales volumes and same-day deliveries. To streamline order fulfillment processes, warehouse managers frequently rely on autonomous mobile robots (AMRs). By supporting human order pickers with AMRs, unproductive picker walking time can be reduced, which is essential for increasing the performance of a traditional picker-to-parts system. We study an AMR-assisted picker-to-parts system in which the warehouse is partitioned into disjoint zones, each with one order picker assigned to it. A set of customer orders is given, each associated with a due date until which its items are to be collected. Each AMR of a given fleet is responsible for transporting the items of a single batch of customer orders from the picking area to the depot. An order picker collects those batch items that are stored in her zone and transports them to a handover location. The AMR visits the handover locations and returns to the depot after collecting all batch items. The objective is to minimize the total tardiness of all customer orders. We define the resulting problem as mixed integer program and provide an efficient heuristic solution approach. Furthermore, we investigate the influence of increasing the AMR fleet size and varying travel and walking speed ratios between AMRs and order pickers. We find that a slight increase in the speed ratio leads to a larger reduction of the total tardiness compared to increasing the AMR fleet size.

1. Introduction

Order picking is the process of retrieving inventory items from their storage locations to satisfy customer orders. It has long been identified as one of the most laborious and costly warehouse operation, accounting for up to 65% of total warehouse operating costs (Coyle, Bardi, & Langley, 2002; Petersen & Schmenner, 1999). Because inadequate order picking performance results in customer dissatisfaction due to long delivery times and increased labor cost for the warehouse, order picking is considered a crucial factor for the competitiveness of a supply chain (Wäscher, 2004).

De Koster, Le-Duc, and Roodbergen (2007) and Napolitano (2012) estimate that about 80% of all order picking systems in Western Europe follow the traditional picker-to-parts setup, in which order pickers walk through the warehouse to retrieve the requested items from their storage locations. While the investment required for such systems is rather low, the major drawback is the large fraction of unproductive picker walking time, which amounts to 50% and more of total order picking time (De Koster et al., 2007; Tompkins, White, Bozer, & Tanchoco, 2010). Although technologies to automate order picking exist (see, e.g., Azadeh, de Koster, & Roy, 2019), warehouse managers rely on human order pickers because of their inherent flexibility and ability to adapt to changes in real-time (Grosse, Glock, & Ballester-Ripoll, 2014).

The steadily increasing global retail sales volumes (Statista, 2019), however, force warehouse managers to improve the performance of their warehouse operations to fulfill customer requirements (e.g., responsive order fulfillment) and to gain advantages over competitors. Here, warehouse managers often face the following challenges:

* Corresponding author.
E-mail addresses: zulj@uni-hohenheim.de (I. Žulj), salewski@wiwi.uni-kl.de (H. Salewski), dominik.goeke@lhsystems.com (D. Goeke), schneider@dpo.rwth-aachen.de (M. Schneider).

https://doi.org/10.1016/j.ejor.2021.05.033
0377-2217/

Fig. 1. The figure depicts SSI Schäfer's FTS 2PICK™ (SSI Schäfer, 2016).

- Tight delivery schedules because of contractually agreed or promised next- or same-day deliveries put an increasing burden on warehouse operations and lead to highly time-critical order fulfillment processes (Boysen, de Koster, & Weidinger, 2019a).
- E-commerce warehouses receive a large number of customer orders which only consist of a few items each (Boysen, Stephan, & Weidinger, 2019b). Traditional picker-to-parts systems are rarely suitable for these requirements. For example, in picker-to-parts systems that are operated with a pick-by-order strategy, customer orders are picked individually on a single order picking tour starting from the depot, proceeding along the storage locations defined by the respective customer order, and ending at the depot. Due to the low pick density per order picking tour in such scenarios, the part of unproductive work that an order picker spends on each customer order while walking through the warehouse is often large. The resulting loss of throughput makes it difficult to meet the customers' expectations for fast delivery (Boysen et al., 2019a).
- On the contrary, in specific branches (e.g., in the online grocery sector), customer orders are likely to consist of dozens of items (Valle, Beasley, & da Cunha, 2017). Handling large customer orders within tight delivery schedules is difficult for warehouse managers because such customer orders require longer picking times compared to small customer orders.
- The rapid spread of SARS-CoV-2 puts an increasing burden on warehouse managers, which are forced to reduce the risk of spreading the virus to avoid a complete stop of the workflow due to infected order pickers: because SARS-CoV-2 is spread person-to-person through close contact and order pickers work close to each other, the risk of virus transmission is very high.

A novel warehousing concept to streamline order fulfillment processes is autonomous mobile robot-assisted (AMR-assisted) order picking, which is addressed in this paper.

1.1. AMR-assisted order picking

An AMR is a computer-controlled and wheel-based load carrier, which automatically moves in its operating environment by a combination of software and sensor-based system. Contrary to automated guided vehicles (AGVs), which follow prescribed paths, AMRs automatically reroute and proceed along alternative paths in case of congestion or obstacles (Azadeh, Roy, & de Koster, 2020). Fig. 1 depicts an AMR handling multiple bins, one for each customer order.

The concept of AMR-assisted order picking has the following advantages:

- *Reduced walking and increased throughput*: Compared to the order picking process in a traditional picker-to-parts system, the AMR support allows order pickers to continuously retrieve items because they remain in the picking area without returning to the depot each time a picking order is completed. Löffler, Boysen, and Schneider (2021) show that compared to an order picking system without the support of AMRs, a reduction of travel distance of approximately 20% can be achieved through AMR usage. Thus, the throughput of completed customer orders can be increased.
- *Easy implementation*: Another advantage is that AMR-assisted order picking can easily be implemented into existing warehousing systems without extensively redesigning the system.
- *High reliability*: AMR systems are highly reliable, i.e., if one AMR is disrupted, the other AMRs are not affected and can continue to operate.
- *High flexibility*: Compared to other concepts based on fixed hardware (e.g., conveyors or automated storage and retrieval systems), AMR-based systems are highly flexible, e.g., when redesigning the warehouse layout (by adding or removing picking aisles). This also enables a quicker adaption to varying workloads by temporarily adding (or removing) AMRs, e.g., when the number of customer orders increases due to season sales.
- *Wide range of applications*: AMRs can be used in many other application areas, such as relocating items in the warehouse or collecting empty containers that are used to supply materials to production.

Disadvantages of AMR-assisted order picking systems concern the investment and regular operating costs for the AMR fleet. However, we believe that the additional investments for AMRs are quickly amortized by the reduction of unnecessary trips from the picking area back to the depot.

1.2. Contributions and organization of the paper

This paper introduces the AMR-assisted order picking problem (AOPP), which is inspired by a warehouse of a logistics center for technical development of a German automotive original equipment manufacturer. More than half a million test components are stored in this logistics center, from which all pre-series centers and prototype constructions are supplied with the required test components. The logistics center receives numerous customer orders, each associated with a due date until which the items of the customer order are to be collected. The customer orders are batched to increase order picking efficiency. Moreover, to avoid unnecessary trips from the picking area of the warehouse back to the central depot, order pickers are supported by a fleet of AMRs. An AMR collects the items of a single batch and transports them from the picking area to the depot.

The warehouse is partitioned into disjoint zones, each comprising a single picking aisle with one order picker assigned to it. Note that order picking systems in which one picking aisle is divided into a single or even multiple zones have also been studied in the literature (see, e.g., Le-Duc & de Koster, 2005) and can be found in real-world warehouses as well. For example, at True Value company, an American wholesaler, the small-item order picking area is divided into single-aisle zones (Frazelle, 2020). In this paper, an order picker collects those items of a batch that are stored in her[1] zone and transports them to a handover location, where she passes the items to an AMR. We assume parallel (synchronized) zoning, i.e., the items of a picking order are simultaneously collected in the zones. Zone picking may speed up order picking because an order picker does not travel between the zones, i.e., each order picker only traverses a smaller area of the warehouse. Moreover, the risk of infection spread between order pickers is reduced because order pickers do not work close to each other. An AMR is equipped with bins at the central depot. Each bin is associated with the items of a single customer order to avoid consolidation effort after picking. An AMR autonomously drives to a handover location, displays the batch ID, and waits until the order picker passes the items and confirms the pick. Then, the order picker returns to her zone to retrieve the items of the next batch, and the AMR drives to other handover locations or it returns to the central depot if all items of the batch have been picked. Here, the shipping of the customer orders is prepared, and the AMR is again equipped with empty bins so that it can process the next batch.

The AOPP decides on the grouping of customer orders into batches and the sequence in which batches should be processed such that the total tardiness of all customer orders, i.e., the extent to which the due dates are violated, is minimized. To the best of our knowledge, the AOPP is novel and has not been explored in the literature so far, although it is highly relevant in practice.

Our contributions can be summarized as follows:

- We propose a mixed integer programming formulation of the AOPP.
- The AOPP is NP-hard because it extends the order batching and sequencing problem (OBSP) introduced by Henn and Schmid (2013). Therefore, we focus on the development of an efficient solution approach to provide solutions for large problem instances. To this end, we propose a two-stage heuristic consisting of an adaptive large neighborhood search (ALNS) component for batching customer orders and an adaption of the well-known Nawaz, Enscore, and Ham (1983) heuristic (called NEH heuristic) for sequencing the batches. We denote our solution approach as ALNS/NEH. The motivation for designing an ALNS is its convincing performance on combinatorial optimization problems like order batching problems. Moreover, previous works demonstrate that ALNS provides robust solution quality for problem instances with different characteristics. This is due to the integrated adaptive weight adjustment. We decided to use the NEH algorithm because we found in preliminary studies that the NEH algorithm performed significantly better compared to other heuristics, such as that of Palmer (1965), Sokolizin (1958), and Petrow (1964). To keep runtimes low, we propose different metrics to efficiently evaluate changes to a batching solution of our problem.
- On newly generated large instances, we analyze the effect of three algorithmic components, i.e., a simulated-annealing-based (SA-based) acceptance criterion, destroy operators, and repair operators, and we show that each component contributes significantly to the solution quality. We design a set of small instances to compare the performance of ALNS/NEH to those of the optimization software IBM ILOG CPLEX Optimizer (CPLEX). The results clearly demonstrate the ability of ALNS/NEH to find high-quality solutions for small instances within very short runtimes.
- To provide managerial insights, we conduct several computational experiments examining the effect of increasing the AMR fleet size and varying travel and walking speed ratios between AMRs and order pickers on the objective of minimizing the total tardiness of all customer orders. The experiments indicate that a slight increase in the speed ratio or the fleet size results in large reductions of the total tardiness. Moreover, we calculate the waiting times of order pickers and AMRs, and we show that the given warehousing system is not prone to long waiting times of order pickers and AMRs.

The remainder of this paper is structured as follows. In Section 2, we review the related literature. In Section 3, we introduce the AOPP and present a mixed integer program. Our ALNS/NEH is detailed in Section 4. Section 5 is devoted to extensive computational experiments. Finally, Section 6 concludes this paper.

2. Literature review

In this section, we review the literature on AMR-assisted order picking and on (variants of) order batching problems (OBPs) integrating the sequencing of batches, which are closely related to the AOPP.

AMR-assisted order picking has so far only been addressed by Azadeh et al. (2020) and Löffler et al. (2021). Azadeh et al. (2020) investigate an AMR-assisted order picking system in which AMRs collaborate with order pickers to fulfill customer orders. The authors propose a queuing network model to estimate the effect of different zoning strategies on the performance of their order picking system. Löffler et al. (2021) consider a warehousing system in which an order picker is accompanied by an AMR during the order picking process. The authors study the routing of AMR-assisted order pickers with both a given and an open processing sequence of incoming customer orders. Their objective is to minimize the travel length for collecting the items of a set of customer orders. To solve the problem with a given processing sequence of customer orders, the algorithm of Ratliff and Rosenthal (1983) is extended and repeatedly solved as an integrated part of a dynamic programming approach. In numerical studies, the authors demonstrate that their procedure outperforms the traveling salesman problem solvers Concorde and Lin-Kernighan-Helsgaun. To address the problem with open processing sequence of customer orders, the authors propose a greedy insertion heuristic.

A variant of an OBP integrating the sequencing of batches is presented in Henn and Schmid (2013). The authors present an iterated local search and attribute-based hill climber algorithm to solve the problem with the objective of minimizing the total tardiness for a set of customer orders. Variants of OBPs integrating the sequencing of batches and the assignment of batches to order pickers are studied in Hong, Johnson, and Peters (2012), Henn (2015), and Matusiak, de Koster, and Saarinen (2017). Hong et al. (2012) consider an integrated order batching, batch sequencing and batch assignment problem. Their problem assumes multiple order pickers operating in narrow picking aisles, in which congestion in the form of picker blocking may occur. The objective of their problem is to minimize the total retrieval time, which consists of travel time, pick time, and congestion delays caused by picker blocking. The authors develop a mixed integer programming approach,

[1] For the sake of readability, we have decided to speak exclusively of female order pickers. Of course, an order picker can also be of any other gender.

which is able to cope with small instances and an SA-based approach for solving larger instances. Henn (2015) presents an integrated order batching, batch assignment and sequencing problem, in which customer orders are grouped to batches, batches assigned to order pickers, and batches scheduled such that the total tardiness is minimized. The problem is solved by means of a variable neighborhood descent and a variable neighborhood search approach. Matusiak et al. (2017) study an order batching, batch sequencing and batch assignment problem for an arbitrary warehouse layout with multiple end depots integrating individual differences in the picking skills of order pickers. The objective of their problem is the minimization of total batch execution times of the order pickers. The combined problem is solved using an ALNS.

Variants of OBPs integrating the sequencing of batches and a picker routing problem (PRP) are addressed in Won and Olafsson (2005), Tsai, Liou, and Huang (2008), and Chen, Cheng, Chen, and Chan (2015). Won and Olafsson (2005) study an integrated order batching, batch sequencing, and PRP for an arbitrary warehouse layout. The objective is to minimize the total tour length and throughput time of all batches. The authors propose a construction heuristic, which first groups customer orders into batches and then sequences the batches. A 2-opt heuristic is used to address the PRP. Tsai et al. (2008) investigate an OBP integrating batch sequencing and picker routing. In their problem, the total costs (depending on the total travel time) are minimized, and both earliness and tardiness are penalized. Contrary to many problems described in the literature, splitting of customer orders is allowed. Thus, items of a single customer order can be collected on different order picking tours. To solve the problem, the authors propose a multiple genetic algorithm, which consists of a genetic algorithm for order batching and batch sequencing as well as a genetic algorithm for the PRP. Chen et al. (2015) propose a non-linear mixed integer programming approach to model the decisions on order batching, batch sequencing, and routing of a single order picker with the objective of minimizing the total tardiness of all customer orders. A genetic algorithm is applied to the order batching and batch sequencing problem, and ant colony optimization is applied to solve the PRP. Finally, Scholz, Schubert, and Wäscher (2017) introduce a mathematical model and a variable neighborhood descent algorithm for their order batching, batch assignment and sequencing problem integrating a PRP.

To conclude, our literature review shows that due-date-related objectives are only studied in few papers, although in many real-world applications, customers expect to receive their requested items within a certain time period. The main differences between our AOPP and the existing literature are as follows:

- In the AOPP, multiple order pickers are responsible for picking a batch of customer orders, whereas the approaches from the literature assume that a batch is collected by a single order picker, i.e., contrary to the OBSP, in the AOPP, we have to coordinate order picking operations between different order pickers.
- The OBSPs from the literature do not consider a zone picking policy.
- In contrast to the OBSP, order pickers do not have to return to the depot in the AOPP. The processing of a subsequent batch can already be started after the items of a previous batch have been handed over to an AMR.

3. The AMR-assisted order picking problem

In this section, we provide a detailed description of the AOPP (see Section 3.1) and present a mathematical model formulation (see Section 3.2).

3.1. Problem description

The AOPP considers a rectangular single-block warehouse with parallel closed-end picking aisles of equal length and width that are connected by one orthogonal cross aisle at the front of the picking aisles (see Fig. 2). Items are stored in storage locations of equal size that are arranged on the left and right side of each picking aisle. Each item is available from exactly one storage location, and each storage location stores exactly one item type. There is a handover location in the cross aisle at the entry of each picking aisle. Let $V = \{1, \ldots, n\}$ denote the set of picking aisles. Each handover location is identified through the associated picking aisle $i \in V$. Handover locations and picking aisles are numbered in ascending order from the leftmost handover location, represented by 1, to the rightmost handover location, represented by n. The depot is located in the cross aisle below the leftmost handover location and associated with the virtual picking aisles 0 and $n+1$. To indicate that V contains the respective instance of the depot, V is subscripted with 0 and/or $n+1$, i.e., $V_0 = V \cup \{0\}$, $V_{n+1} = V \cup \{n+1\}$, and $V_{0,n+1} = V \cup \{0\} \cup \{n+1\}$.

The items of a set of customer orders O have to be collected from the picking area of the warehouse. A customer order $o \in O$ is specified by a non-empty set of order lines. Each order line indicates a particular item and the requested quantity of this item. Moreover, each customer order $o \in O$ is associated with a due date d_o until which the items contained in the customer order must be handed over at the depot for packing and shipping to the respective customer. Customer orders can be grouped into batches, for which we make the following assumptions:

- *No splitting of customer orders*: The items of a customer order cannot be distributed among different batches because splitting may result in unacceptable sorting effort. Note that by defining the subsets into which customer orders can be split (this may also be single items) as the new customer orders, our AOPP model could still address the scenario in which splitting of customer orders is allowed.
- *Batch size*: The number of customer orders that may be contained in a batch is restricted by the carrying capacity C of an AMR. The capacity is expressed by the number of items which can be carried by the AMR. Parameter c_o denotes the number of items contained in customer order $o \in O$.
- *Generation of batches*: The set of batches is represented by B. In order not to restrict the batching decision, we set $|B| = |O|$. Thus, we include the extreme case in which each customer order can be defined as a single batch. We use binary variables v_{bo} to indicate whether customer order $o \in O$ is included in batch $b \in B$ ($v_{bo} = 1$) or not ($v_{bo} = 0$).

With respect to the retrieval of the requested items from their storage locations, we make the following modeling assumptions:

- *Zone picking*: We consider a zone picking system in which the requested items of a batch are simultaneously retrieved by multiple order pickers in multiple zones. Each zone is assigned a single order picker, who is responsible for retrieving those items of a batch which are stored in the picking aisle of her zone. A zone comprises a single picking aisle and the associated handover location. If the items of a batch are stored in different picking aisles, different order pickers are involved in picking these items.
- *Equipment of order pickers*: Each order picker is initially positioned at her handover location and equipped with a picking cart with multiple bins as well as a pick list. The bins are used to temporarily store the retrieved items of a batch, and each bin is dedicated to a single customer order contained in that batch. The pick list of an order picker specifies the selected batches

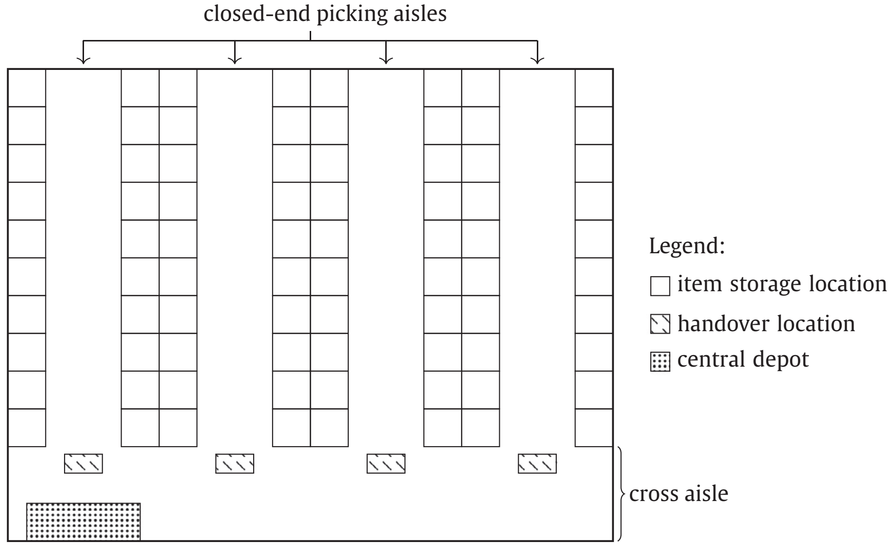

Fig. 2. Warehouse layout assumed in our AOPP.

to be processed in her zone and their processing sequence. Recall that an order picker only handles a single batch at a time. For each of these batches, the pick list indicates (i) the items to be retrieved, (ii) the requested quantities of these items, (iii) the item retrieval sequence, and (iv) the assignment of each of these items to its corresponding bin on the picking cart. In the following, we detail the modeling of the batch processing sequence and the routing of order pickers.

- *Batch processing sequence for order pickers*: The batch processing sequence is described by precedence relationships for all pairs of batches $b, d \in B$. To model the batch processing sequence, we introduce binary decision variables z_{bd}, which denote whether the order pickers process batch $b \in B$ before batch $d \in B$ ($z_{bd} = 1$) or not ($z_{bd} = 0$).
- *Retrieval strategy*: The order picker is guided by the following routing scheme when picking the batch items stored in her picking aisle: Starting from the handover location, she enters the picking aisle and walks to the farthest storage location in which a requested item is stored. Here, she retrieves the requested number of items and places them in the associated bins. All other picking locations in this picking aisle are successively visited on the way back to the handover location. Given the above described setting of the order picking system with closed-end picking aisles and a single cross aisle, an order picker is optimally routed (concerning the objective of minimizing travel time) in this way. To model the retrieval sequence, we introduce the continuous decision variables s_{bi}, and the parameters w_{oi} and r_{oi}. Variables s_{bi} describe the order picker's start time at handover location $i \in V$ for picking the batch items stored in picking aisle $i \in V$. Parameters w_{oi} contain (i) the time the order picker spends on walking from her handover location $i \in V$ to the farthest picking location and from the last visited picking location back to handover location $i \in V$ and (ii) the time she requires to walk between the picking locations of customer order $o \in O$ in picking aisle $i \in V$. Parameters r_{oi} specify the time for retrieving the requested items from their storage locations and placing them into the appropriate bins on the picking cart. Upon collection of the batch items stored in picking aisle $i \in V$, the order picker returns to her handover location, where she passes the items to an AMR.

The AMR-assistance in the considered order picking system is based on the following decisions and modeling assumptions:

- *AMR fleet*: A fleet of m identical AMRs is initially located at the depot.
- *Equipment of AMRs*: Exactly one AMR collects the picked items of a single batch from the relevant handover locations (i.e., those from which the items of a batch are to be collected) on a single tour through the warehouse. To this end, an AMR is equipped with multiple bins for temporarily storing the batch items. Each of the bins is intended for the items of a single customer order contained in a batch. We assume that (i) all requested items of a customer order fit into a single bin, and (ii) the batch is created such that the AMR capacity is sufficient to carry all items of the batch.
- *AMR tours*: An AMR tour starts from the depot, proceeds along the cross aisle to the relevant handover locations, and ends at the depot. On an AMR tour, each of the handover locations of a batch is visited exactly once. Because of safety reasons, an AMR may only change the travel direction along the cross aisle once, i.e., at the rightmost handover location from which batch items are to be collected, and all other relevant handover locations have to be visited on the way to this handover location. Passing of other AMRs is allowed. To model an AMR tour, we introduce auxiliary parameters η_{oi}, which state whether handover location $i \in V$ has to be visited by the AMR handling customer order $o \in O$ ($\eta_{oi} = 1$) or not ($\eta_{oi} = 0$). To determine the relevant handover locations for each batch, we define binary variables γ_{bi}, which take value 1 if the handover location $i \in V$ has to be visited by the AMR handling batch $b \in B$, and 0 otherwise. Let parameters t_{ij} denote the travel time from depot/handover location $i \in V_0$ to handover location/depot $j \in V_{n+1}$. Travel times are assumed to be deterministic and symmetric, i.e., $t_{ij} = t_{ji}$. Continuous decision variables a_{bi} denote the arrival time of the AMR collecting the items of batch $b \in B$ at handover location $i \in V$.
- *Assignment of batches to AMRs and batch processing sequence by AMRs*: In the AOPP, we also decide on the assignment of batches to AMRs and the sequence according to which the batches assigned to an AMR are to be processed by the AMR. Both decisions are implicitly modeled by defining precedence relationships for all pairs of batches $b, d \in B$. To this end, we define that a batch has either a direct predecessor batch or no direct predecessor batch if it is the first batch processed by a certain AMR. To model the batch processing sequence, we introduce binary decision variables ζ_{bd} and α_b. Variables ζ_{bd} de-

note whether the items of batch $b \in B$ are collected directly before those of batch $d \in B$ by a certain AMR ($\zeta_{bd} = 1$) or not ($\zeta_{bd} = 0$). Variables α_b take value 1 if batch $b \in B$ has no predecessor and thus is processed first by one of the AMRs, and 0 otherwise.

As described above, an order picker returns to her handover location $i \in V$ after picking the batch items stored in picking aisle $i \in V$. Here, she immediately places the picked items into the appropriate bins of the AMR that is handling batch $b \in B$ (called associated AMR) if the AMR has already arrived at her handover location $i \in V$, or she waits until it arrives. To model handovers, we use the following notation. Continuous decision variables h_{bi} denote the start time of handing over the items of batch $b \in B$ at handover location $i \in V$ to the associated AMR. The associated times for the handovers of the items of customer order $o \in O$ are given by parameters l_{oi}.

After an order picker has handed over the items to the AMR, she returns to her picking aisle to process the next batch according to the pick list. The AMR moves either towards other relevant handover locations, or it returns to the depot if all items of the batch that is currently handled by the AMR are collected. Upon arrival at the depot, it takes u_o time units to unload the items of customer order $o \in O$. After unloading the items of each customer order contained in a batch, the batch is then considered as completed, and the AMR is equipped with (new) bins for collecting the items of the next batch. Continuous decision variables f_b describe the completion time of batch $b \in B$.

We measure the quality of a solution to the AOPP in terms of the total tardiness of all customer orders τ because customer orders have to be completed until given due dates to avoid delays in shipments to customers or in production. Continuous decision variables τ_o define the tardiness of customer order $o \in O$ as the positive difference between the time the customer order is completed and its due date. Note that the completion time of a customer order corresponds to the completion time of the batch in which the customer order is contained.

To provide a compact overview of the AMR-assisted order picking process described above, Fig. 3 summarizes the process from the perspective of an order picker and Fig. 4 from the perspective of an AMR.

Finally, Fig. 5 demonstrates precedence relationships between two order pickers and a single AMR when processing a single batch in the considered system. Consider a batch which consists of items that are stored in both picking aisles of the given warehouse. The order picker assigned to the left picking aisle starts from her handover location at time 0 and returns with the picked items from her picking aisle at time 4. The other order picker starts at time 0 and returns at time 8. The associated AMR starts from the depot at time 0 and first proceeds along the cross aisle to the left handover location, which it reaches at time 2. As soon as the respective order picker arrives at her handover location, she starts handing over the picked items to the AMR already waiting at the handover location. After the handover is completed at time 6, the AMR travels to the right handover location, which it reaches at time 8. The order picker assigned to the right picking aisle also arrives at her handover location at time 8, and she hands over the picked items to the AMR. Finally, the AMR returns to the depot and reaches it at time 16. Here, the unloading of the collected items starts and finishes at time 22.

3.2. Mathematical model formulation

In the following, we present a mathematical model formulation for the AOPP. Using the notation summarized in Table 1, the AOPP can be formulated as the following mixed integer problem.

$$\text{minimize} \sum_{o \in O} \tau_o \tag{1}$$

subject to

$$\sum_{b \in B} \upsilon_{bo} = 1 \qquad \forall o \in O \tag{2}$$

$$\sum_{o \in O} \upsilon_{bo} \cdot c_o \leq C \qquad \forall b \in B \tag{3}$$

$$\sum_{o \in O} \upsilon_{bo} \cdot \eta_{oi} \geq \gamma_{bi} \qquad \forall b \in B;\, i \in V_{0,\,n+1} \tag{4}$$

$$\sum_{o \in O} \upsilon_{bo} \cdot \eta_{oi} \leq M \cdot \gamma_{bi} \qquad \forall b \in B;\, i \in V_{0,\,n+1} \tag{5}$$

$$z_{bd} + z_{db} = 1 \qquad \forall b, d \in B;\, b \neq d \tag{6}$$

$$\alpha_d + \sum_{\substack{b \in B \\ b \neq d}} \zeta_{bd} \geq \gamma_{d,\,0} \qquad \forall d \in B \tag{7}$$

$$\sum_{\substack{d \in B \\ d \neq b}} \zeta_{bd} \leq \gamma_{b,\,0} \qquad \forall b \in B \tag{8}$$

$$\sum_{b \in B} \alpha_b \leq m \tag{9}$$

$$h_{bi} + \sum_{o \in O} \upsilon_{bo} \cdot l_{oi} - M \cdot (1 - z_{bd}) \leq s_{di} \qquad \forall b, d \in B;\, b \neq d;\, i \in V \tag{10}$$

$$s_{bi} + \sum_{o \in O} \upsilon_{bo} \cdot r_{oi} + \upsilon_{bo'} \cdot w_{o'i} \leq h_{bi} \qquad \forall b \in B;\, o' \in O;\, i \in V \tag{11}$$

$$a_{bi} \leq h_{bi} \qquad \forall b \in B;\, i \in V \tag{12}$$

$$h_{bi} + \sum_{o \in O} \upsilon_{bo} \cdot l_{oi} + t_{ij} - M \cdot (2 - \gamma_{bi} - \gamma_{bj}) \leq a_{bj} \qquad \forall b \in B;\, i \in V_0;\, j \in V;\, j > i \tag{13}$$

$$f_b + t_{0,\,i} - M \cdot (2 - \gamma_{di} - \zeta_{bd}) \leq a_{di} \qquad \forall b, d \in B;\, b \neq d;\, i \in V \tag{14}$$

$$h_{bi} + \sum_{o \in O} \upsilon_{bo} \cdot (l_{oi} + u_o) + t_{i,\,n+1} - M \cdot (1 - \gamma_{bi}) \leq f_b \qquad \forall b \in B;\, i \in V_{n+1} \tag{15}$$

$$f_b - d_o - M \cdot (1 - \upsilon_{bo}) \leq \tau_o \qquad \forall b \in B;\, o \in O \tag{16}$$

$$a_{bi}, f_b, h_{bi}, s_{bi}, \tau_o \geq 0 \qquad \forall b \in B;\, o \in O;\, i \in V_{0,\,n+1} \tag{17}$$

$$z_{bd}, \alpha_b, \gamma_{bi}, \zeta_{bd}, \upsilon_{bo} \in \{0, 1\} \qquad \forall b, d \in B;\, o \in O;\, i \in V_{0,\,n+1} \tag{18}$$

The objective of minimizing the total tardiness of all customer orders is defined in (1). Constraints (2) guarantee that each customer order is assigned to exactly one batch. Constraints (3) ensure that the capacity of an AMR is not exceeded. Constraints (4) and (5) determine the relevant handover locations for each batch. Note that M denotes a sufficiently large positive number, which must be at least as large as the maximum completion time over all batches.

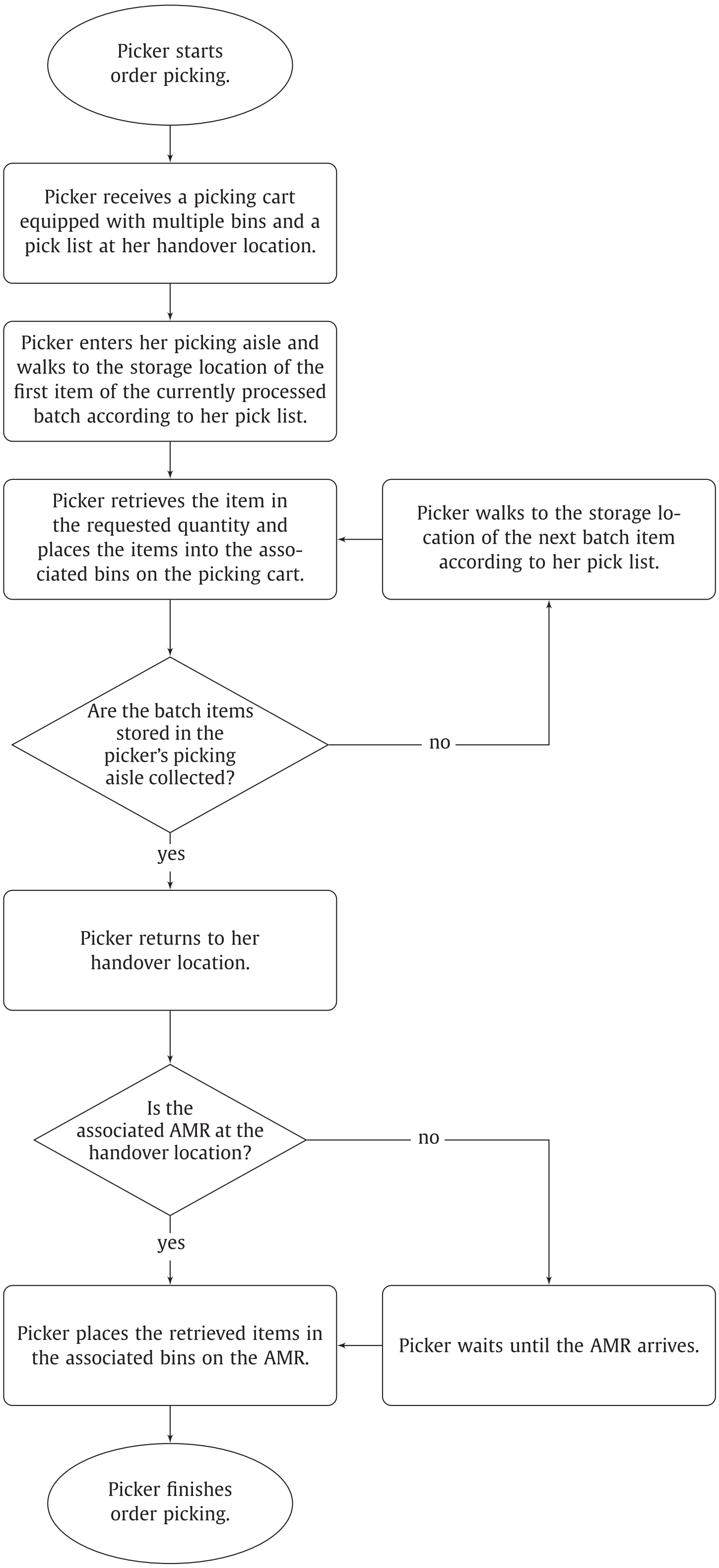

Fig. 3. Order picking process from the perspective of an order picker in the AMR-assisted order picking system.

However, because the completion time of a batch is not known before solving a specific test instance, we calculate M based on the worst case scenario in which batching of customer orders is not allowed and customer orders are sequentially picked, i.e., it is not possible to process several customer orders simultaneously. Then, M can be calculated as follows:

$$M = \sum_{o \in O} \left(\max_{i \in V} \{r_{oi} + w_{oi}\} + \sum_{i \in V} l_{oi} + u_o + 2 \cdot \max_{i,\, j \in V_{0,n+1}} \{t_{ij}\} \right) \qquad (19)$$

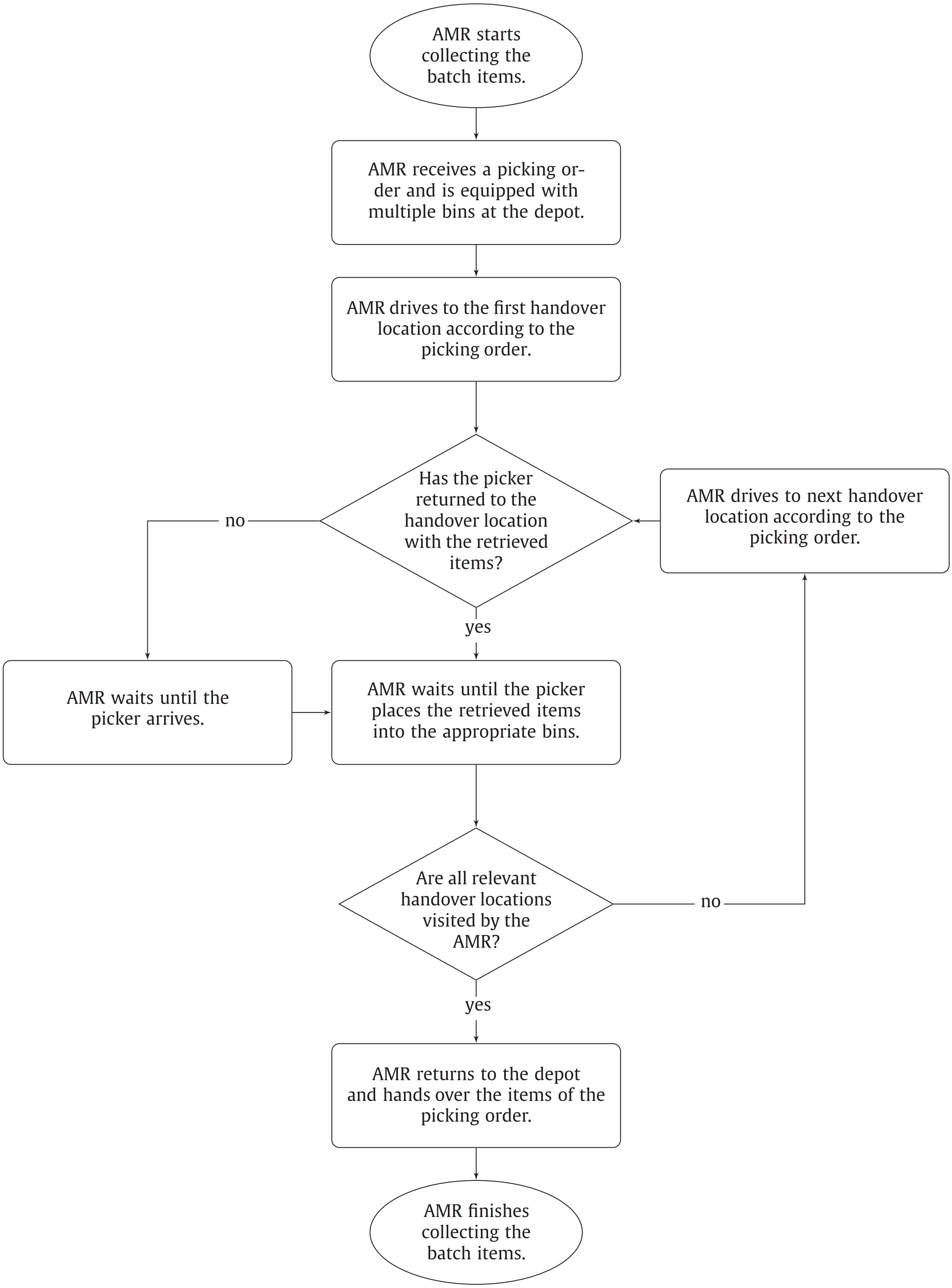

Fig. 4. Order picking process from the perspective of an AMR in the AMR-assisted order picking system.

Constraints (6) define the sequence according to which the batches are to be processed by the order pickers. The sequence in which the batches are processed by AMRs is modeled in constraints (7), (8), and (9). Constraints (7) enforce that each selected batch is either the first batch or a direct predecessor batch of another batch handled by the same AMR. Constraints (8) assure that each selected batch has at most one direct successor batch. Constraint (9) states that at most one batch can be the first batch handled by a single AMR.

Constraints (10) define the order picker's start time of picking the batch items stored in her picking aisle. If a batch $d \in B$ is not the first batch processed by the order picker, we link the

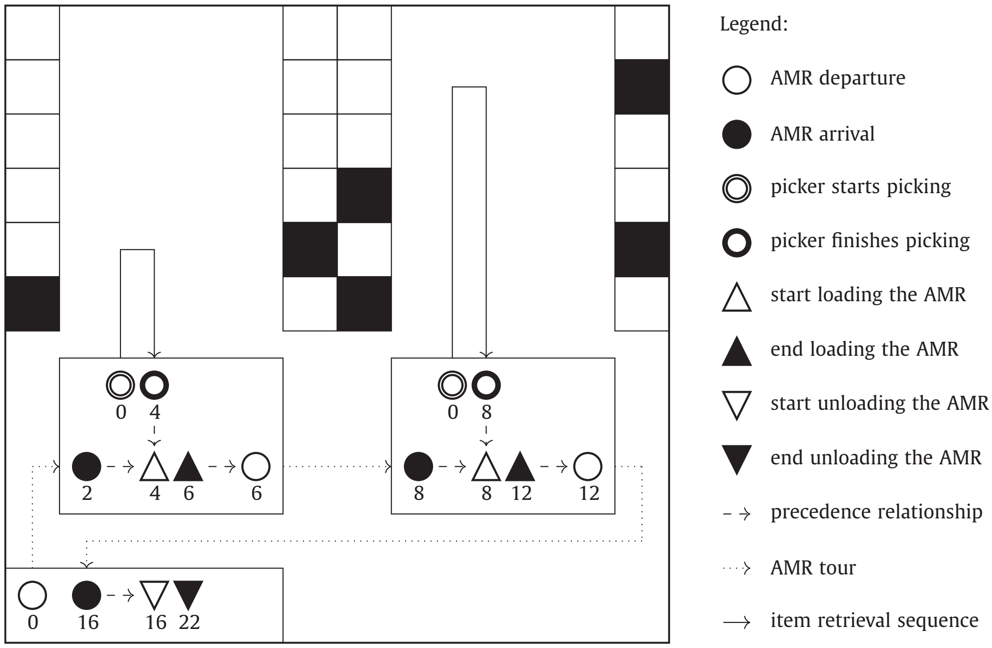

Fig. 5. An example illustrating precedence relationships between two order pickers and a single AMR when processing a single batch. The black rectangles represent picking locations defined by the batch.

Table 1
Overview of the notation used in the AOPP model.

$0, n+1$	instances of the depot represented by the virtual picking aisles 0 and $n + 1$
Sets	
B	set of batches with $\|B\| = \|O\|$ (indices: b, d)
O	set of customer orders (indices: o, o')
V	set of picking aisles with each handover location $i \in V$ identified through the associated picking aisle $i \in V$ (indices: i, j)
V_0	set of depot instance 0 and picking aisles with $V_0 = V \cup \{0\}$ (indices: i, j)
V_{n+1}	set of depot instance $n + 1$ and picking aisles with $V_{n+1} = V \cup \{n+1\}$ (indices: i, j)
$V_{0,n+1}$	set of depot instances and picking aisles with $V_{0,n+1} = V \cup \{0\} \cup \{n+1\}$ (indices: i, j)
Parameters	
c_o	number of items contained in customer order $o \in O$
C	capacity of an AMR expressed by the number of items which can be carried by the AMR
d_o	due date of customer order $o \in O$
l_{oi}	time the order picker requires to pass the items of customer order $o \in O$ at handover location $i \in V$ to the associated AMR
m	number of AMRs
M	a sufficiently large positive number
r_{oi}	time the order picker requires to retrieve the items of customer order $o \in O$ which are stored in picking aisle $i \in V$
t_{ij}	time an AMR requires to travel from depot/handover location $i \in V_0$ to depot/handover location $j \in V_{n+1}$
u_o	time required to unload the items of customer order $o \in O$ from the associated AMR
w_{oi}	time the order picker spends on walking to retrieve the items of customer order $o \in O$ which are stored in picking aisle $i \in V$
η_{oi}	take value 1 if depot/handover location $i \in V_{0,n+1}$ has to be visited by the AMR handling customer order $o \in O$; 0 otherwise
Continuous decision variables	
a_{bi}	arrival time of the AMR handling batch $b \in B$ at handover location $i \in V$
f_b	completion time of batch $b \in B$
h_{bi}	order picker's start time of passing the items of batch $b \in B$ to the associated AMR at handover location $i \in V$
s_{bi}	order picker's start time of picking the items of batch $b \in B$ at handover location $i \in V$
τ_o	tardiness of customer order $o \in O$
Binary decision variables	
α_b	take value 1 if batch $b \in B$ is the first batch handled by a certain AMR; 0 otherwise
γ_{bi}	take value 1 if the depot/handover location $i \in V_{0,n+1}$ has to be visited by the AMR handling batch $b \in B$; 0 otherwise
υ_{bo}	take value 1 if customer order $o \in O$ is included in batch $b \in B$; 0 otherwise
z_{bd}	take value 1 if batch $b \in B$ is handled before batch $d \in B$ by the order pickers; 0 otherwise
ζ_{bd}	take value 1 if batch $b \in B$ is handled directly before batch $d \in B$ by a certain AMR; 0 otherwise

order picker's start time of picking the items of batch $d \in B$ to the time at which the order picker has handed over the items of the direct predecessor batch $b \in B$ to the associated AMR. Constraints (11) and (12) guarantee that the order picker cannot hand over the picked items to the appropriate AMR until (i) she has returned to the handover location with these items (see constraints (11)), and (ii) the AMR has arrived at her handover location (see constraints (12)). Constraints (13) determine the arrival time of the AMR handling batch $b \in B$ at each handover location $j \in V$. Moreover, in combination with constraints (4) and (5), the constraints state that an AMR collecting the items of a batch starts from the depot, drives to the relevant handover locations from the leftmost to the rightmost, and then returns to the depot. It is also assured that the relevant handover locations are visited exactly once on each AMR tour. Constraints (14) link the arrival time of the AMR handling batch $d \in B$ at handover location $i \in V$ to the completion time of batch $b \in B$ (plus the time the AMR requires to travel from the depot to handover location $i \in V$) if the following holds: (i) batch $b \in B$ is handled by the same AMR as batch $d \in B$, and (ii) batch $b \in B$ is processed before batch $d \in B$ by the AMR. Constraints (15) compute the completion time for each batch $b \in B$, and constraints (16) calculate the tardiness for each customer order $o \in O$. Finally, the continuous decision variables and the binary decision variables are defined in constraints (17) and (18), respectively.

4. Solution approach

In this section, we present our ALNS/NEH solution approach. Section 4.1 describes a surrogate objective function used to efficiently evaluate changes to a batching solution of our problem. In Section 4.2, we introduce the ALNS component for grouping customer orders into batches. Subsequently, the generated batches are sequenced by using an NEH-based heuristic (see Section 4.3). The batches are assigned to the next available AMR while respecting the given batch processing sequence.

4.1. Surrogate objective function

Calculating the objective function value of a complete AOPP solution or changes in the objective function value caused by modifications are time-consuming. To illustrate this, we use the example presented in Fig. 5. We assume that the batch consists of two customer orders, $o = 1$ and $o = 2$. The items of customer order $o = 1$ are stored in the left picking aisle and those of customer order $o = 2$ in both picking aisles. Let us consider an operator which removes customer order $o = 1$ from the given batch and thus causes the following changes: The time the order picker spends on retrieving the items from the left picking aisle is reduced by two time units, and the time to hand over these items to the associated AMR decreases by one time unit (compared to the scenario given in Fig. 5). The order picker reaches the handover location at time 2, and she starts handing over the items. After the handover is completed, the AMR drives to the right handover location and arrives there at time 5. Accordingly, the start and end time of the subsequent order picking activities change (except the retrieval of the items from the right picking aisle). Even this small example shows that a slight modification of the solution requires many recalculations to evaluate the objective function. Obviously, such recalculations increase in the case of multiple batches affected by the modification.

While testing our algorithm, we identified the following metrics that serve as a surrogate objective function but are less time-consuming to evaluate:

- ξ_b denotes the surrogate completion time of batch b, which is computed in the same way as the completion time f_b (see Section 3), except that for the computation of ξ_b, b is considered to be the only batch that has to be processed by the order pickers and the associated AMR.
- φ_b corresponds to the surrogate tardiness of batch b. We compute φ_b as follows:

$$\varphi_b = \sum_{o \in O_b} \max\{0; \xi_b - d_o\}. \tag{20}$$

  O_b denotes the set of customer orders contained in batch b, and d_o defines the due date of customer order o.
- η_s specifies the surrogate total tardiness of all batches $b \in B_s$ in batching solution s, which is computed as follows:

$$\eta_s = \sum_{b \in B_s} \varphi_b. \tag{21}$$

4.2. Adaptive large neighborhood search for batching customer orders

This section details our ALNS component for grouping customer orders into batches, which is based on the ALNS presented in Žulj, Kramer, and Schneider (2018) for the standard order batching problem. Large neighborhood search (LNS), introduced by Shaw (1997), iteratively destroys and repairs solutions by removing and reinserting a relatively large number of customer orders. ALNS, originally proposed by Ropke and Pisinger (2006a), is an extension of LNS using multiple destroy and repair operators within the same search process, which are chosen based on their performance in past iterations. The probability with which an operator is selected is dynamically updated during the search.

In Algorithm 1, a pseudocode overview of the ALNS component

Algorithm 1 Overview of the ALNS algorithm.

```
1:  generate initial batching solution s
2:  s* ← s, w_i^- = (10, ..., 10), w_i^+ = (10, ..., 10)
3:  repeat
4:      randomly draw the number of q^- customer orders to be removed
5:      choose destroy and repair operators (h_i^-, h_i^+) according to
        probabilities (π_i^-, π_i^+)
6:      s' ← h_i^+(h_i^-(s))
7:      determine batch processing sequence for s' by using the NEH
        algorithm
8:      assign the batches to the next available AMR while respecting
        the given batch processing sequence
9:      if acceptSA(s') then
10:         s ← s'
11:     end if
12:     if η_{s'} < η_{s*} then
13:         s* ← s'
14:     end if
15:     adjust the weights w_i^- and w_i^+ as well as the probabilities
        π_i^- and π_i^+ of the destroy and repair operators
16: until stopping criterion is met
17: return s*
```

is given. First, we generate an initial batching solution s by using an adaption of the Clarke and Wright algorithm (line 1), referred to as C&W(ii) algorithm (see Section 4.2.1). Then, we define s as the best known solution s^*, and we initialize the weights of the destroy and repair operators w_i^- and w_i^+ (line 2). Initially, all operators are assigned the same weight. Next, s is improved by several ALNS iterations (see Section 4.2.2), in which neighboring batching solutions are accepted according to an SA-based acceptance criterion (see Section 4.2.3). To this end, we randomly draw the number of q^- customer orders to be removed (line 4) and choose a pair of destroy and repair operators h_i^- and h_i^+ according to probabilities π_i^- and π_i^+ (line 5) to generate a new solution s' (line 6). Then, we determine the batch processing sequence for s' by using an adaption of the NEH algorithm (line 7), which is detailed in Section 4.3. The batches are assigned to the next available AMR while respecting the given batch processing sequence (line 8). An improving solution is always accepted, and a new deteriorating solution s' is accepted if it satisfies the SA-based acceptance criterion (lines 9–14). Then, we adjust (i) the weights w_i^- and w_i^+ of the chosen destroy and repair operators based on a scoring system and (ii) the selection probabilities π_i^- and π_i^+ of the operators, which are determined by the weights (line 15). This procedure is repeated until the stopping criterion is met (line 16). Finally, the best batching solution s^* is returned (line 17). In the following sections, we provide further details on the ALNS component and the NEH component.

4.2.1. Initial solution

We generate the initial batching solution by implementing an adaption of the C&W(ii) algorithm (Elsayed & Unal, 1989). To minimize the total tardiness of all customer orders in problem settings with loose due dates, it may be reasonable to sort customer orders in descending order of their due date and then group them until the capacity of an AMR is no longer sufficient (see, e.g., Scholz et al., 2017). However, in problem settings with tight due dates, not only the similarity of the due dates of the customer orders should be considered when generating batches. In particular, generating batches with similar customer orders (i.e., similar storage locations that have to be visited) positively affects the minimization of the total tardiness of all customer orders. Therefore, we group customer orders into batches dependent on (i) the similarity of their due date and (ii) the saving in terms of the surrogate completion time reduction, which results from processing customer orders simultaneously instead of separately. Our procedure for generating an initial batching solution can be described as follows:

- Each customer order in the considered problem instance is initially assigned to a single batch.
- We compute relatedness measure ϕ_{bd} for each pair of batches b and d for which the AMR carrying capacity is sufficient as follows:

$$\phi_{bd} = \frac{\xi_b + \xi_d - \xi_{bd}}{\max\{\varepsilon; \max\limits_{b\in B}\{\xi_b\} - \min\limits_{b\in B}\{\xi_b\}\}} \cdot \frac{\min\{\delta_b; \delta_d\}}{\max\{\varepsilon; \max\{\delta_b; \delta_d\}\}}, \tag{22}$$

  where ξ_{bd} denotes the surrogate completion time when processing batches b and d simultaneously, parameters δ_b (δ_d) represent the minimum due date among all customer orders which are contained in batch b (d), and ε specifies a small positive number. Consequently, the term $\xi_b + \xi_d - \xi_{bd}$ describes the surrogate completion time saving that results from picking both batches b and d simultaneously instead of separately. We normalize the surrogate completion time saving using the extreme values of the surrogate completion times across the set of all batches, which are given by the problem instance, and we weight them with the quotient of $\min\{\delta_b;\ \delta_d\}$ and $\max\{\varepsilon;\ \max\{\delta_b;\ \delta_d\}\}$ to stimulate the grouping of batches with similar due dates. Larger values of ϕ_{bd} denote a larger relatedness between two customer orders.
- Subsequently, the pairs of batches are sorted in non-ascending order of ϕ_{bd}. Our algorithm starts with the pair of batches with the largest relatedness measure. The following three cases may occur while grouping a pair of batches (b, d) to a larger batch: (i) a new batch is generated if neither of the two batches b and d is yet assigned to a larger batch, (ii) if one of the batches b and d is already assigned to a larger batch, the other batch is assigned to the same batch, provided that the AMR carrying capacity is sufficient, (iii) the next pair of batches is considered if both of the batches b and d are already assigned to larger batches or if the AMR carrying capacity is not sufficient. We recalculate the relatedness measure ϕ_{bd} after each grouping of batches to larger batches.

Subsequently, we improve the resulting initial batching solution by means of an ALNS.

4.2.2. Destroy and repair operators

Each destroy operator removes q^- customer orders from an existing batching solution s. To remove customer orders from the current batching solution, ALNS uses the following three destroy operators:

Random removal randomly removes customer orders from the current batching solution until q^- customer orders are removed.

Worst removal aims at removing customer orders which appear to be unfavorably assigned to batches in the current batching solution s with respect to the surrogate total tardiness η_s. To this end, we compute for each customer order the reduction of the surrogate total tardiness η_s when removing customer order o from s. Then, we sort all customer orders in a list of size L in descending order of the surrogate total tardiness reduction. At each iteration, we choose the customer order at position $\lfloor L \cdot u^p \rfloor$ in this list, where u denotes a uniformly distributed number in $[0, 1)$ and $p \geq 1$ a randomization parameter, which allows to balance the influence of randomness on the selection of customer orders and thus avoids repeatedly removing the same customer orders. We repeat this procedure until q^- customer orders are removed.

Relatedness removal proposed by Shaw (1997) removes customer orders that are related to each other according to several criteria and thus are likely to be easily interchangeable. If related customer orders are removed, there are more possibilities for reinsertion, and thus new, potentially better batching solutions can be found. Otherwise, if customer orders are removed that are very different from each other, they will probably be reinserted into their original batches due to unattractive reinsertion possibilities. To define the relatedness between two customer orders, we use the relatedness measure introduced in Section 4.2.1. The first customer order to be removed is randomly chosen. All remaining customer orders are sorted in a list in descending order of their relatedness to the selected customer order. Then, we choose the customer order at position $\lfloor L \cdot u^p \rfloor$ in this list of size L, where again u denotes a uniformly distributed number in $[0, 1)$ and p a randomization parameter, which allows to balance the influence of randomness and relatedness on the selection of the customer orders to be removed. Afterwards, we randomly choose a customer order from those already removed from the batching solution, and we repeat the described procedure until q^- customer orders are removed.

To reinsert the previously removed customer orders into the current batching solution, ALNS uses one of the following three repair operators:

Random insertion aims at solution diversification. To this end, random insertion selects an arbitrary batch and inserts the customer order into it.

Greedy insertion iteratively assigns customer orders to the batch with minimal increase in surrogate total tardiness η_s of batching solution s.

Regret insertion proposed by Ropke and Pisinger (2006b) anticipates the future effect of an insertion operation and thus improves the greedy insertion operator. In the last iterations of the ALNS, greedy insertion tends to insert those customer orders whose insertion leads to a large increase in the surrogate total tardiness η_s of batching solution s because there are not many attractive possibilities to insert the customer orders. This is due to the fact that a relatively large number of customer orders is already contained in the batches. The k-regret value of each customer order o describes the difference in the value of the surrogate total tardiness η_s when inserting the customer order in its k-best batch instead of its best batch. The customer order with the largest k-regret value is selected for insertion into the best batch, and the procedure is repeated until all customer orders are inserted. We implement the 2-regret and 3-regret heuristic. In preliminary studies, we found that values of $k > 3$ did not improve the solution quality.

4.2.3. Adaptive mechanism

The adaptive weight adjustment procedure dynamically evaluates the importance of each operator during the search process. In this context, the selection probability π_i of an operator $i \in \mathcal{X}$ is modified based on the performance of the operator in previous iterations. Initially, all operators are assigned the same weight ω. The performance of an operator $i \in \mathcal{X}$ is measured in terms of a scoring system. Let o_i denote the score value of operator $i \in \mathcal{X}$. If a previous destroy-repair operation results in a new best batching solution, we increase the current scores of the respective operators by o_{best}, if a better batching solution is found by o_{imp}, and if a new deteriorating batching solution is generated but accepted according to the SA-based acceptance criterion by o_{acc}. The search process of the ALNS component is divided into a number of segments of γ iterations. After γ iterations, the new weight of operator $i \in \mathcal{X}$ is calculated as $w_{i+1} = (1-r)\cdot w_i + r\cdot \frac{o_i}{\beta_i}$, where w_i corresponds to the weight of operator $i \in \mathcal{X}$. The parameter $r \in [0, 1]$ controls the reaction speed of the weight adjustment and takes the success of an operator in previous segments into account. The number of times that an operator $i \in \mathcal{X}$ was chosen in the previous segment is denoted by β_i. Afterwards, π_i is calculated according to $\pi_i = w_i \;/\; \sum\limits_{i \in \mathcal{X}} w_i$ (see Ropke & Pisinger, 2006a).

4.2.4. Acceptance criterion

To diversify the search, we use an acceptance criterion that is inspired by SA. While solutions that improve the current solution are always accepted, we accept a new deteriorating solution s' with probability $e^{-(\eta_{s'}-\eta_s)/t_i}$, where $t_i > 0$ denotes the current temperature. We set the starting temperature t_0 to a value such that a solution that deteriorates the current solution by a% is accepted with a probability of 50%. Each time a deteriorating solution is accepted, we decrease the temperature by a constant factor c such that in the last 20% of iterations the temperature is smaller than 0.0001.

4.3. NEH-based heuristic for deciding the batch processing sequence

We propose an adaption of the NEH algorithm introduced by Nawaz, Enscore, and Ham (1983) to determine the batch processing sequence. The idea of our batch sequencing algorithm is that a batch b which has a larger surrogate tardiness φ_b should be given higher priority and be processed before another batch d with less surrogate tardiness φ_d. In Algorithm 2, we give an overview of the proposed algorithm, which is described in the following:

Algorithm 2 Overview of the NEH-based algorithm.

```
1: input: batching solution s
2: for each batch b ∈ B_s contained in the batching solution s calculate its surrogate tardiness φ_b
3: generate sequence λ := (λ_1, ..., λ_ι) by sorting the batches in non-ascending order of their surrogate tardiness φ_b
4: π* := (λ_1)
5: for i ← 2 to ι do
6:     insert batch λ_i in the position of π* which minimizes the (partial) total tardiness τ
7: end for
8: return π*
```

- The algorithm starts with batching solution s generated by the ALNS component (line 1).
- Then, we calculate surrogate tardiness φ_b for each batch $b \in B_s$ contained in the batching solution s (line 2).
- Next, we determine sequence $\lambda := (\lambda_1, \ldots, \lambda_\iota)$, in which the batches are arranged by decreasing surrogate tardiness φ_b, e.g., λ_1 denotes the batch with largest surrogate tardiness, and λ_2 indicates the batch with the second largest surrogate tardiness, and so on (line 3).
- In line 4, we select the batch with the largest surrogate tardiness, i.e., λ_1, and we insert it in the partial batch processing sequence π^*, i.e., $\pi^* := (\lambda_1)$.
- Last, for $i = 2$ to ι, we do the following: we insert batch λ_i in the position of π^* (among the i possible ones) which minimizes the (partial) total tardiness τ (lines 5–7). The partial batch processing sequence with the smallest (partial) total tardiness τ is returned (line 8).

5. Numerical studies

This section describes the numerical studies (i) to investigate the influence of the SA-based acceptance criterion as well as the destroy and repair operators used by ALNS/NEH on the solution quality, (ii) to assess the performance of ALNS/NEH in comparison to CPLEX, and (iii) to give managerial insights with respect to AMR-assisted order picking. Because no established testbed is available for our problem, we generate three new sets of instances, which we describe in Section 5.1. Subsequently, the parameter setting of our ALNS and the results of our experiments are presented in Sections 5.2–5.7.

5.1. Generation of benchmark instances

We generate test instances of two different sizes. The set of small instances is denoted by S, and sets M and L comprise larger instances. The instance generation process is described in the following:

- *Warehouse layout*: We consider three different sizes of single-block warehouses with parallel closed-end picking aisles (see Section 3), namely $n = 10$, $n = 15$, and $n = 20$ picking aisles. The warehouse which comprises 10 picking aisles contains 20 storage locations in each picking aisle, 10 on the left and 10 on the right. The warehouse containing 15 picking aisles consists of 30 storage locations per picking aisle (15 on each side of the picking aisle), and in the warehouse with 20 picking aisles, 40 storage locations are arranged in each picking aisle (20 on each side of the picking aisle). Thus, 200 items are stored in the warehouse with $n = 10$ picking aisles, 450 in the warehouse with $n = 15$ picking aisles, and 800 in the warehouse with $n = 20$ picking aisles. The warehouses apply a storage policy that randomly assigns items to storage locations. The depot is located below the leftmost handover location. The instances in set S are based on the warehouse with $n = 10$ picking aisles, those in set M, on $n = 10$, $n = 15$, or $n = 20$ picking aisles, and those in set L, on $n = 15$ or $n = 20$ picking aisles.
- *Physical dimensions of the warehouses*: An order picker has to cover 1.5 length units (LUs) to get from her handover location to the first storage location in the associated picking aisle. The distance between two adjacent storage locations amounts to 1 LU. An AMR moves 1 LU from the depot to the leftmost handover location. The travel distance between two neighboring handover locations is 5 LUs.
- *Time parameters for order picking activities*: The time an order picker requires to retrieve an item from its storage location is 8 seconds, and the time an order picker spends on handing an item over to an AMR amounts to 4 seconds. It takes 4 seconds to unload an item from an AMR at the depot. The traveling speed of AMRs equals the walking speed of order pickers: both move 1 LU in 3 seconds.
- *Number and capacity of picking devices*: The number of AMRs is fixed to $m = 1$ and $m = 2$ in set S, to $m = 4$ in set M, and to $m = 6$ in set L. The carrying capacity C of both AMR and picking

cart is $C = 6$ or $C = 8$ items in set S and $C = 60$ items in sets M and L.

- *Generation of pick lists*: The number of customer orders is $|O| = 5, 6, 7$ in set S, $|O| = 40, 60, 80, 100$ in set M, and $|O| = 40, 50, 60, 70, 80, 90, 100$ in set L. In the small instances, the number of items contained in a customer order is uniformly distributed over the interval $[2, 4]$. In sets M and L, the number of items is randomly drawn from the uniformly distributed interval $[5, 25]$.
- *Generation of due dates*: To generate due dates, we assign to each customer order contained in a problem instance a due date of 0 and solve the resulting AOPP model using ALNS/NEH. The minimum and maximum completion time over the resulting batches define the extreme values of the uniformly distributed interval of due dates, from which we randomly choose a due date for each customer order in the respective problem instance.

Combining the above described parameter values leads to 12 instance groups for set S, to 12 instance groups for set M, and to 14 instance groups for set L. The instance groups associated with set S are identified by the number of customer orders $|O|$, the number of AMRs m, and the capacity C. The groups of larger instances are identified by the number of picking aisles n and the number of customer orders $|O|$. For each instance group, we generate 10 instances, i.e., $12 \cdot 10 = 120$ small instances, $12 \cdot 10 = 120$ instances for set M, and $14 \cdot 10 = 140$ instances for set L. All instances and detailed results of our numerical studies are available for download at https://www.dropbox.com/sh/v50tiy5qbje5zou/AABnX1rdG7iD2Jq_38omrIXja?dl=0.

5.2. Parameter setting for the adaptive large neighborhood search component

We adopt the approach proposed by Ropke and Pisinger (2006a) to tune the parameters of our ALNS component. We start from a base parameter setting that we found during the testing phase of our algorithm. Afterwards, we change the value of a single parameter, fix the rest of the parameters, and we perform five runs on randomly selected instances from the sets of instances described in Section 5.1. The parameter setting with the best average result is chosen as the final setting for the respective parameter, and we repeat this procedure with the next parameter. We randomly determine the order in which the tuning of the parameters is performed. The ALNS component seems relatively insensitive to the variation of parameters because none of the tested parameter settings results in significant deterioration of solution quality.

The final parameter setting, which is used for all computational studies, is the following. The scores used in the adaptive weight adjustment are set to $o_{best} = 120$, $o_{imp} = 100$, and $o_{acc} = 80$. The initial weight of all operators is set to $\omega = 10$. We fix the number of iterations after which the weights of the ALNS component are adjusted to $\gamma = 10$, the reaction factor to $r = 0.4$, the randomization parameter to $p = 3$, and the percentage of deterioration to $a = 25\%$. The number of customer orders removed from the current solution q^- is randomly drawn from the interval $[q_{min}, q_{max}]$ with $q_{min} = 0.175 \cdot |O|$ and $q_{max} = 0.35 \cdot |O|$, where again $|O|$ denotes the number of customer orders. To achieve a good tradeoff between the solution quality and the runtime of our algorithm, we set the number of ALNS iterations to $\kappa = |O|$. Table 2 summarizes the parameter setting of the ALNS component.

All experiments are executed on a desktop computer with an Intel Core i7-3770 Processor at 3.5 GHz, 16 GB of memory, and Windows 7 Professional. The solution method is implemented using Java, and the IBM ILOG CPLEX solver (version 12.9) is applied to solve the mixed integer program presented in Section 3.2.

Table 2
Overview of the parameter setting of the ALNS component used to generate a batching solution.

Parameter	Parameter value
γ	10
q_{min}, q_{max}	$0.175 \cdot \lvert O\rvert$, $0.35 \cdot \lvert O\rvert$
r	0.4
$o_{best}, o_{imp}, o_{acc}$	120, 100, 80
ω	10
p	3
a	25%
κ	$\lvert O\rvert$

5.3. Effect of the simulated annealing-based acceptance criterion

This section analyzes the effect of using an SA-based acceptance criterion after the ALNS phase instead of accepting only improving solutions denoted as ALNS/NEH w/o SA. In Table 3, we give an overview of the results obtained on the large instances in set L and present averages for groups of instances defined by the number of picking aisles (column n) and the number of customer orders (column $|O|$). The reported results are based on five runs. In column BKS, we provide the best known solution as the average of the best objective function values obtained by one of the tested methods for each of the individual instances in the group over the five runs. For ALNS/NEH and ALNS/NEH w/o SA, we give the following information:

- Δ_f (%) denotes the percentage gap between the best solution found by the respective variant and the BKS over the five runs. The percentage gap is computed as $100 \cdot (f_k - \text{BKS})/f_k$, where f_k denotes the best solution achieved by variant $k \in K$. The smallest gap found by any of the variants is indicated in bold.
- t (s) reports the average runtime over the five runs in seconds.

The results show that ALNS/NEH outperforms ALNS/NEH w/o SA with an average gap of the total tardiness of 0.5% to the BKS compared to an average gap of 1.04% obtained by ALNS/NEH w/o SA,

Table 3
Effect of using an SA-based acceptance criterion on the large instances in set L. In the first two columns, we specify the group of instances defined by the number of picking aisles (column n) and the number of customer orders (column $|O|$). In column BKS, we provide the BKS as the average of the best objective function values obtained by one of the tested methods for each of the individual instances in the group. For ALNS/NEH and ALNS/NEH w/o SA, the table reports the percentage gap between the best solution found by the respective variant and the BKS over the five runs (column Δ_f (%)), and the average runtime over the five runs in seconds (column t (s)). The smallest gap found by any of the variants is indicated in bold.

n	$\lvert O\rvert$	BKS	ALNS/NEH		ALNS/NEH w/o SA	
			Δ_f (%)	t (s)	Δ_f (%)	t (s)
15	40	5572	**0.76**	1.6	0.89	1.5
15	50	8594	**0.26**	3.3	0.81	3.0
15	60	10214	**0.53**	6.3	1.77	6.9
15	70	13649	**0.33**	13.8	0.85	12.3
15	80	20442	**0.48**	21.2	1.58	23.6
15	90	22819	**0.85**	35.0	1.13	25.3
15	100	27502	**0.70**	47.9	1.28	46.9
20	40	5648	**0.40**	1.7	0.85	1.7
20	50	7607	**0.40**	3.4	0.98	3.7
20	60	9571	**0.17**	6.5	1.34	6.7
20	70	15430	**0.47**	13.2	0.90	13.9
20	80	20655	**0.35**	28.4	0.75	24.5
20	90	29111	**0.63**	37.4	0.64	33.7
20	100	27658	**0.68**	52.8	0.73	51.4
Average			**0.50**	19.5	1.04	18.2

i.e., using the SA-based acceptance criterion instead of simply accepting improving solutions leads to a larger reduction of the average total tardiness of all customer orders. With respect to runtimes, the table shows that both variants require similar runtimes on average.

5.4. Effect of the destroy and repair operators

In this section, we investigate the effect of the destroy and repair operators. To this end, we use 50 randomly selected large instances of set L. The contribution of each destroy and repair operator is measured by successively deactivating each operator and keeping the remaining operators activated. We then compare the solution quality achieved with the respective configuration to those of our full ALNS/NEH in which all components are active over five runs.

Table 4 summarizes the results of this experiment. We compare the average percentage gaps Δ_f (%) of the best solutions found to the BKS for each configuration. By ✓, we indicate that a destroy/repair operator is active in the respective configuration, and by ✗, we denote that a destroy/repair operator is inactive in the respective configuration. The configuration representing our full ALNS/NEH is given in bold.

The results show that inactivating an operator clearly decreases the solution quality between 2.7% to 6.8% compared to the full algorithm. The strongest effect is observed in case of the relatedness removal operator. Thus, the destroy and repair operators used in the ALNS component seem to be adequate to effectively explore the search space. Note that the runtime is not significantly affected by inactivating an operator.

5.5. Comparison of ALNS/NEH to a commercial solver

This section assesses the performance of ALNS/NEH on the small instances. We compare the performance of ALNS/NEH and CPLEX solving AOPP. We perform five runs of ALNS/NEH on each instance, and we restrict the solution time limit of CPLEX solving AOPP to 1800 seconds.

Table 5 presents aggregate results on the small instances and reports averages for groups of instances defined by the number of customer orders (column $|O|$), the number of AMRs (column m), and the carrying capacity of the picking device (column C). In column BKS, we provide the BKS as the average of the best objective function values obtained with the respective method for each of the individual instances in the group. For ALNS/NEH and CPLEX, we give the following information:

- *#opt* indicates the number of instances solved to optimality. The optimality of a solution is proven by CPLEX.

Table 4
Effect of destroy and repair operators on 50 randomly selected large instances of set L. We compare the average percentage gaps Δ_f (%) of the best solutions found to the BKS for each configuration over five runs. By ✓, we indicate that a destroy/repair operator is active in the respective configuration, and by ✗, we denote that a destroy/repair operator is inactive in the respective configuration.

ALNS operators	Selected operators							
Random removal	✓	✗	✓	✓	✓	✓	✓	✓
Worst removal	✓	✓	✗	✓	✓	✓	✓	✓
Relatedness removal	✓	✓	✓	✗	✓	✓	✓	✓
Random insertion	✓	✓	✓	✓	✗	✓	✓	✓
Greedy insertion	✓	✓	✓	✓	✓	✗	✓	✓
Regret-2 insertion	✓	✓	✓	✓	✓	✓	✗	✓
Regret-3 insertion	✓	✓	✓	✓	✓	✓	✓	✗
Δ_f (%)	**0.0**	2.7	4.2	6.8	2.7	6.7	4.4	4.7

- Δ_f (%) denotes the percentage gap between the best solution found by the respective method and the BKS. The percentage gap is computed as $100 \cdot (f_k - \text{BKS})/f_k$, where f_k denotes the best solution achieved by method $k \in K$.
- t (s) reports the average runtime in seconds.

Although the number of customer orders ranges only between 5 and 7 customer orders, each consisting of 2, 3, or 4 items to be picked, CPLEX is not able to consistently solve the instances to optimality within the given runtime limit. The table shows that CPLEX guarantees an optimal solution for 73 of the 120 instances. On all tested instances, ALNS/NEH is able to match the solution quality of CPLEX in all runs on each instance. Concerning runtime, ALNS/NEH requires significantly less runtime (0.019 seconds on average) than CPLEX (846.3 seconds on average).

The results of ALNS/NEH clearly demonstrate its ability to solve small instances of the AOPP within very short runtimes.

5.6. Experiments on the AMR fleet size and speed

This section investigates the effect of increasing (i) the number of AMRs and (ii) the AMR speed on the average total tardiness. We focus on these two problem parameters because they can be easily adjusted by warehouse managers without extensively redesigning the order picking system. All experiments are based on the instances of set M (see Section 5.1).

We consider three different sizes of the AMR fleet, i.e., $m = 4$, $m = 8$, and $m = 12$ AMRs, and we vary the speed ratio σ from 1.0 to 3.0 in steps of 0.5, where σ defines the quotient of the travel speed of AMRs and the walking speed of order pickers. For each combination of m and σ, we compute the average of the best objective function values obtained by ALNS/NEH for each of the individual instances in the respective instance group over five runs. Recall that each instance group is defined by the warehouse size n and the number of customer orders $|O|$. The resulting values are compared to the scenario (called reference case) with the same warehouse size n and the same number of customer orders $|O|$ but with the AMR fleet size fixed to $m = 4$ and the AMR speed restricted to the walking speed of order pickers, i.e., $\sigma = 1.0$.

For each instance group and pair of m and σ, Table 6 reports the change in average total tardiness in percent compared to the average total tardiness of the respective reference case. Column m denotes the number of AMRs, and the remaining columns are divided into three blocks, where each block reports the results for one of the three warehouse sizes for different speed ratios (columns σ). Additionally, in Figs. 6–8, we illustrate the average total tardiness for all combinations of different numbers of AMRs and speed ratios for the three warehouse sizes. Fig. 6 shows the results for $n = 10$ picking aisles, Fig. 7 for $n = 15$ picking aisles, and Fig. 8 for $n = 20$ picking aisles.

In the following, we discuss the results of our experiments:

Effect of the AMR speed The results show that the AMR speed significantly influences the average total tardiness. For example, for $n = 10$ picking aisles and $m = 4$ AMRs, a marginal increase in the speed ratio from $\sigma = 1.0$ to $\sigma = 1.5$ reduces the average total tardiness over the different numbers of customer orders by 49.5%. This indicates that the AMR travel times account for a substantial share of the time required to complete the customer orders (i.e., order picking time). This value is also due to our comparison scenario with $m = 4$ AMRs in which the bottleneck lies with the AMRs, i.e., there are longer waiting times for the order pickers. The strongest reductions are achieved at the largest speed ratio of $\sigma = 3.0$: for example, for $n = 15$ picking aisles, $m = 4$ AMRs the average total tardiness over the different numbers of customer orders decreases by 88.2%. However, the results show that with increasing speed ratio, the additional reduction of the average total tardiness tends to

Table 5
Performance of ALNS/NEH in comparison to CPLEX on small instances in set S. In the first three columns, we specify the group of instances defined by the number of customer orders (column $|O|$), the number of AMRs (column m), and the carrying capacity of the picking device (column C). In column BKS, we provide the BKS as the average of the best objective function values obtained with the respective method for each of the individual instances in the group. For CPLEX and ALNS/NEH, the table reports the number of instances solved to optimality (column #opt), the percentage gap between the best solution found by the respective method and the BKS (column Δ_f (%)), and the average runtime in seconds (column t (s)).

				CPLEX			ALNS/NEH		
$\|O\|$	m	C	BKS	#opt	Δ_f (%)	t (s)	#opt	Δ_f (%)	t (s)
5	1	6	212	10	0.0	28.8	10	0.0	0.027
5	1	8	175	10	0.0	21.4	10	0.0	0.016
5	2	6	137	10	0.0	11.8	10	0.0	0.009
5	2	8	144	10	0.0	10.7	10	0.0	0.010
6	1	6	788	1	0.0	1744.9	1	0.0	0.021
6	1	8	229	8	0.0	987.5	8	0.0	0.014
6	2	6	133	10	0.0	600.3	10	0.0	0.014
6	2	8	121	10	0.0	268.6	10	0.0	0.024
7	1	6	287	1	0.0	1620.6	1	0.0	0.029
7	1	8	295	0	0.0	1800.0	0	0.0	0.014
7	2	6	268	3	0.0	1260.6	3	0.0	0.019
7	2	8	314	0	0.0	1800.0	0	0.0	0.027
Average					0.0	846.3		0.0	0.019
Minimum					0.0	10.7		0.0	0.009
Maximum					0.0	1800.0		0.0	0.029

Table 6
Change in objective function values (in percent). For each instance group (defined by the number of customer orders $|O|$ and the warehouse size n) and pair of m and σ, we report the change in average total tardiness in percent compared to the average total tardiness of the respective reference case. A reference case is identified by the warehouse size n, the number of customer orders $|O|$, $m = 4$ AMRs, and $\sigma = 1.0$, and it is indicated in bold.

		$n = 10$					$n = 15$					$n = 20$				
$\|O\|$	m	σ=1.0	σ=1.5	σ=2.0	σ=2.5	σ=3.0	σ=1.0	σ=1.5	σ=2.0	σ=2.5	σ=3.0	σ=1.0	σ=1.5	σ=2.0	σ=2.5	σ=3.0
40	4	**0.0**	−55.5	−74.6	−80.9	−81.5	**0.0**	−68.3	−85.2	−92.8	−93.1	**0.0**	−82.2	−94.5	−98.2	−98.6
40	8	−49.6	−69.1	−77.9	−81.2	−81.9	−62.6	−81.0	−89.9	−93.1	−94.2	−76.1	−91.7	−97.6	−98.2	−98.6
40	12	−51.2	−71.0	−78.5	−81.8	−82.7	−62.9	−81.4	−90.0	−93.2	−94.7	−76.4	−94.0	−97.6	−98.5	−98.8
60	4	**0.0**	−46.7	−59.0	−66.1	−74.4	**0.0**	−66.2	−81.2	−89.2	−90.2	**0.0**	−82.3	−93.8	−95.2	−96.9
60	8	−46.6	−63.0	−71.3	−71.8	−75.2	−75.7	−86.4	−92.5	−92.6	−93.1	−86.0	−92.0	−94.4	−96.7	−97.9
60	12	−48.2	−63.9	−71.5	−72.1	−75.3	−75.9	−88.1	−92.6	−92.7	−93.6	−86.7	−93.8	−96.1	−97.0	−98.1
80	4	**0.0**	−49.5	−65.2	−70.5	−70.6	**0.0**	−63.7	−81.7	−83.9	−88.9	**0.0**	−72.7	−87.3	−90.2	−93.2
80	8	−56.8	−71.9	−72.7	−75.8	−77.1	−76.1	−86.0	−88.7	−89.9	−90.7	−86.9	−92.7	−94.4	−94.6	−94.7
80	12	−57.7	−72.7	−73.9	−76.0	−77.2	−78.9	−86.1	−89.3	−90.2	−90.9	−87.5	−92.8	−94.4	−94.7	−95.3
100	4	**0.0**	−46.4	−53.2	−58.5	−63.7	**0.0**	−55.3	−69.2	−78.3	−80.6	**0.0**	−68.5	−86.6	−91.5	−91.9
100	8	−60.1	−62.2	−65.0	−66.6	−68.8	−76.5	−82.5	−83.9	−84.5	−86.1	−89.6	−93.9	−94.0	−95.1	−95.3
100	12	−60.8	−64.1	−65.2	−68.5	−69.1	−77.4	−83.4	−84.1	−85.0	−86.7	−89.8	−95.1	−95.2	−95.4	−95.5
Average	4	0.0	−49.5	−63.0	−69.0	−72.6	0.0	−63.4	−79.3	−86.0	−88.2	0.0	−76.4	−90.5	−93.8	−95.1
Average	8	−53.3	−66.5	−71.7	−73.8	−75.8	−72.7	−84.0	−88.8	−90.0	−91.0	−84.6	−92.6	−95.1	−96.1	−96.6
Average	12	−54.5	−67.9	−72.3	−74.6	−76.1	−73.8	−84.7	−89.0	−90.2	−91.4	−85.1	−93.9	−95.8	−96.4	−96.9

decline. For example, for $n = 20$ picking aisles and $m = 4$ AMRs, an increase in the speed ratio from $\sigma = 1.0$ to $\sigma = 1.5$ reduces the average total tardiness over the different numbers of customer orders by 76.4% and from $\sigma = 1.0$ to $\sigma = 3.0$ by 95.1%.

We make the following observations when comparing the results obtained for different warehouse sizes and numbers of customer orders:

- *Warehouse size*: The larger the warehouse, the more distance has to be covered by both the order pickers and the AMRs to complete a given set of customer orders. Therefore, an increase in the speed ratio has the strongest effect in the case of the largest warehouse. For example, on the instances with $n = 10$ picking aisles and $m = 4$ AMRs, an increase in the speed ratio from $\sigma = 1.0$ to $\sigma = 1.5$ reduces the average total tardiness over the different numbers of customer orders by 49.5%, while on the instances which assume $n = 20$ picking aisles, the reduction amounts to 76.4%.
- *Number of customer orders*: When comparing the results obtained for a small number of customer orders to those for a large number of customer orders, we observe that a slight increase in the AMR speed tends to have a larger impact on the average total tardiness assuming a small number of customer orders. For example, for $n = 15$ picking aisles, $|O| = 40$ customer orders, and $m = 4$ AMRs, an increase in the speed ratio from $\sigma = 1.0$ to $\sigma = 1.5$ reduces the average total tardiness by 68.3%, while for $|O| = 100$ customer orders, a reduction of 55.3% is achieved. To understand this rather non-intuitive result, we examined the solutions obtained on the respective instances in more detail and found the following: On the instances with a small number of customer orders, the share of the AMR travel times (and thus the impact on the average total tardiness) is significantly larger compared to other components of the order picking time. As a result, increasing the AMR speed leads to a relatively large reduction of the average total tardiness. However, with an increasing number of customer or-

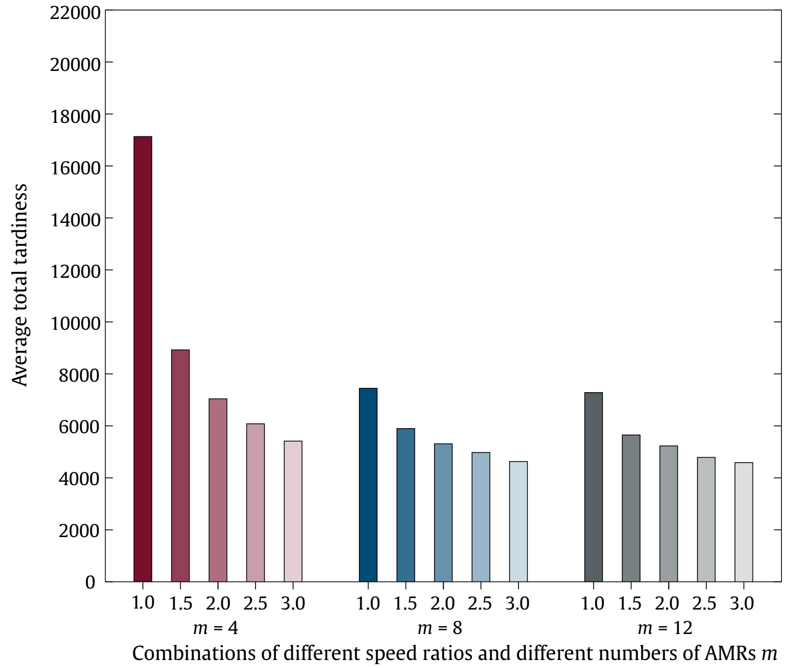

Fig. 6. Average total tardiness that results on the instances assuming a warehouse with 10 picking aisles. We depict the average total tardiness for all combinations of different speed ratios and numbers of AMRs.

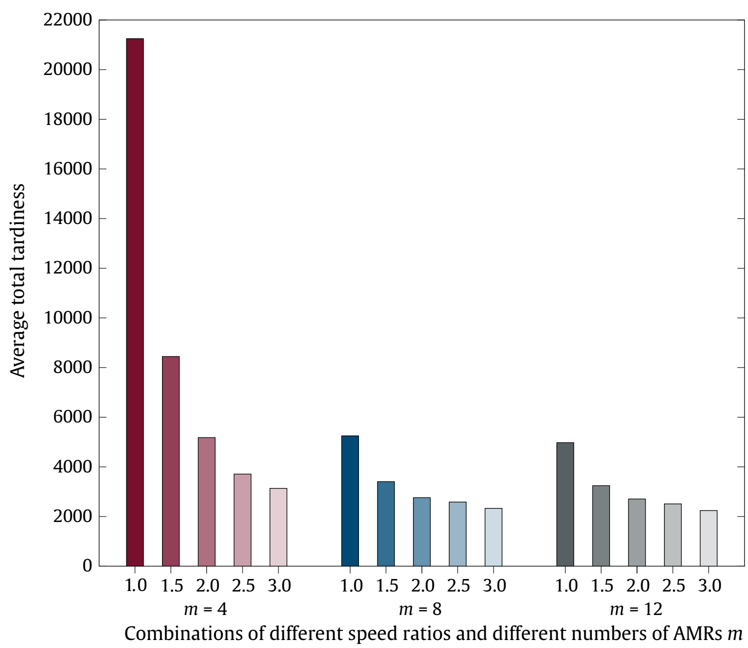

Fig. 7. Average total tardiness that results on the instances assuming a warehouse with 15 picking aisles. We depict the average total tardiness for all combinations of different speed ratios and numbers of AMRs.

ders, the impact of AMR travel times weakens, while the impact of the other components increases. Thus, in the case of a large number of customer orders, an increase in the AMR speed has only a smaller effect on reducing the average total tardiness.

Effect of the number of AMRs The larger the number of AMRs, the larger the reduction of the average total tardiness for the investigated AMR fleet sizes. However, the reported values show that expanding the AMR fleet from $m = 4$ to $m = 8$ leads to a similar reduction as from $m = 4$ to $m = 12$: for example, for the largest warehouse and a speed ratio of $\sigma = 1.0$, Table 6 reports that a doubling of the AMR fleet size from $m = 4$ to $m = 8$ reduces the average total tardiness over the different numbers of customer orders by 84.6%, while by increasing the AMR fleet size from $m = 4$ to $m = 12$, a reduction of 85.1% is achieved.

We observe the following when comparing the results with regard to the different warehouse sizes and numbers of customer orders:

- *Warehouse size*: In the largest warehouse, an increase in the AMR fleet size has a stronger effect on the average total tardiness compared to the results which are obtained in the case of the smallest warehouse as shown in Figs. 6 and 8. Again, this might be due to the large distances that have to be covered by

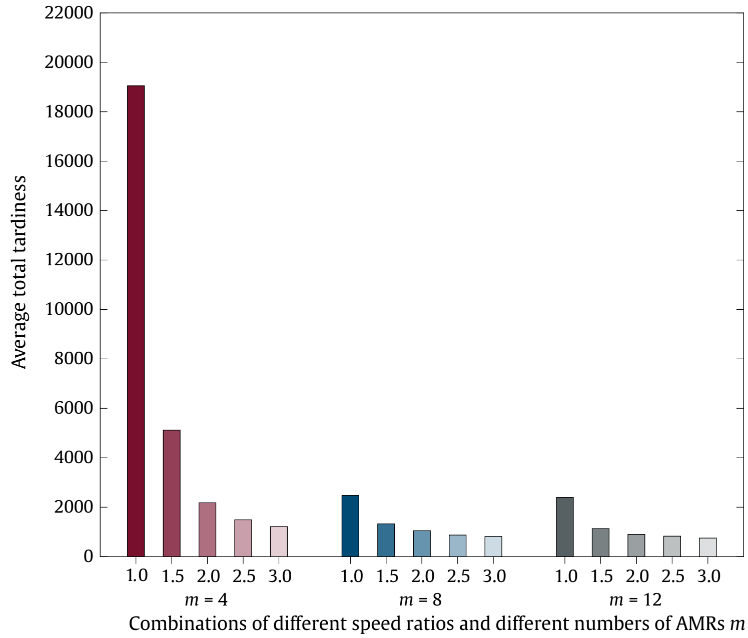

Fig. 8. Average total tardiness that results on the instances assuming a warehouse with 20 picking aisles. We depict the average total tardiness for all combinations of different speed ratios and numbers of AMRs.

the AMRs in the largest warehouse. For example, in the smallest warehouse and for a speed ratio of $\sigma = 1.0$, an increase in the AMR fleet size from $m = 4$ to $m = 8$ reduces the average total tardiness over the different numbers of customer orders by 53.3%. In the largest warehouse, the reduction amounts to 84.6%.

- *Number of customer orders*: In the case of a small number of customer orders, an expansion of the AMR fleet from $m = 4$ to $m = 8$ offers less potential for reduction in comparison to the results obtained on the instances with a large number of customer orders. For example, on the instances with $n = 10$ picking aisles, $|O| = 40$ customer orders, and a speed ratio of $\sigma = 1.0$, a doubling of the number of AMRs from $m = 4$ to $m = 8$ reduces the average total tardiness by 49.6%, while on those with $|O| = 100$ customer orders, a reduction of 60.1% is realized. We have examined this in more detail and found that for smaller numbers of customer orders, there are already enough AMRs to achieve a good solution quality so that expanding the AMR fleet size hardly affects the average total tardiness.

Overall managerial insights An interesting result is observed on the instances assuming $|O| = 40$ customer orders: In all warehouses, a marginal increase in the speed ratio of 0.5 is more advantageous than doubling the number of AMRs from $m = 4$ to $m = 8$. For example, for $n = 10$ picking aisles (and $|O| = 40$ customer orders), an increase in the speed ratio from $\sigma = 1.0$ to $\sigma = 1.5$ leads to a reduction of 55.5% of the average total tardiness, while expanding the AMR fleet from $m = 4$ to $m = 8$ (and fixing the speed ratio to $\sigma = 1.0$) reduces the average total tardiness by 49.6%. In the case of $m = 8$ AMRs, this observation does not only hold true for $|O| = 40$ but also for $|O| = 60, 80, 100$ customer orders. Here, an increase in the speed ratio of 0.5 results in a larger reduction of the average total tardiness compared to increasing the number of AMRs from $m = 8$ to $m = 12$ AMRs. For example, in the warehouse with $n = 15$ picking aisles, increasing the speed ratio from $\sigma = 1.0$ to $\sigma = 1.5$ (assuming $m = 8$ AMRs) reduces the average total tardiness over all customer orders by 84.0%, while an expansion of the AMR fleet from $m = 8$ to $m = 12$ results in a reduction of 73.8%.

As can be seen from Figs. 6–8, a fleet size of $m = 8$ AMRs and a speed ratio of $\sigma = 1.5$ seem to be a reasonable combination to reduce the average total tardiness in our computational experiments. In real-world warehouses, the traveling speed of AMRs is restricted to or is only slightly faster than the walking speed of order pickers due to safety reasons (see, e.g., Löffler et al., 2021). Thus, speed ratios between $\sigma = 1.0$ and $\sigma = 1.5$ are the norm.

5.7. Experiments on the waiting time of order pickers and AMRs

This section aims to provide details on the waiting time of order pickers and AMRs. To this end, we use the instances that assume $n = 10$ picking aisles, $|O| = 100$ customer orders, and a carrying capacity of $C = 60$ items. We then consider two different sizes of the AMR fleet, i.e., $m = 4$ and $m = 8$, and two speed ratios $\sigma = 1.0$ and $\sigma = 1.5$. For each combination, we compute (i) the average time order pickers are waiting for the AMRs and (ii) the average time AMRs are waiting for the order pickers over the individual instances in the respective instance group. Fig. 9 illustrates the results for order pickers by using box plots for different numbers of AMRs m and speed ratios σ and Fig. 10 for AMRs. The length of the whiskers is computed as $1.5 \cdot IQR$, where IQR denotes the interquartile range.

The results presented in Fig. 9 show that the fraction of order picker waiting time of the total order picking time is negligible. The larger the number of AMRs and the speed ratio, the shorter the waiting time. Even if only $m = 4$ AMRs and a speed ratio of $\sigma = 1.5$ are assumed, the waiting time of order pickers is approximately only 2.31%. Contrary to what might be expected, the results indicate that the considered warehousing system is not prone to long waiting times of order pickers.

With respect to the waiting time of the AMRs, it can be observed that the larger the number of AMRs and the speed ratio, the longer the waiting time. The box plots show average waiting times of 0.32% (for $\sigma = 1.0$) and 1.07% (for $\sigma = 1.5$) in the case of $m = 4$ AMRs and 14.43% (for $\sigma = 1.0$) and 18.17% (for $\sigma = 1.5$) in the case of $m = 8$ AMRs. However, the experiments on the AMR

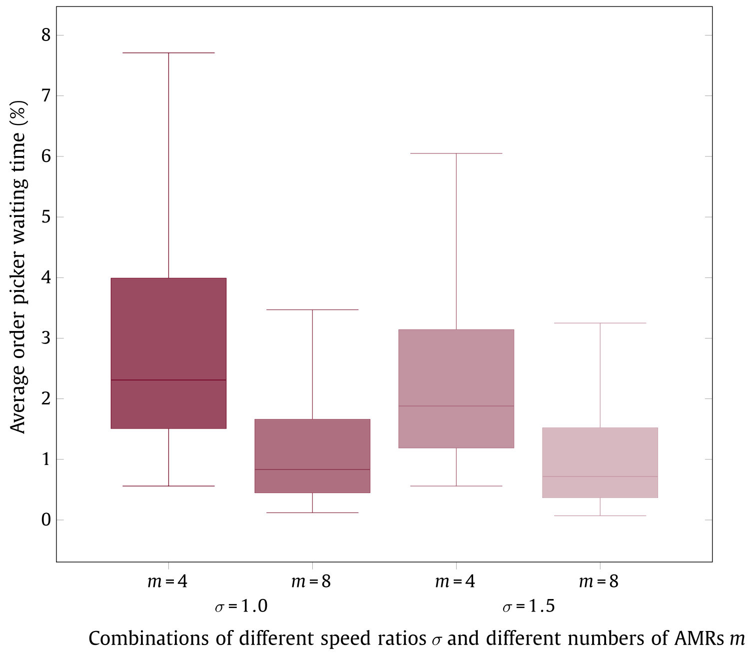

Fig. 9. Average waiting time of the order pickers for the AMRs in percent for different numbers of AMRs and speed ratios.

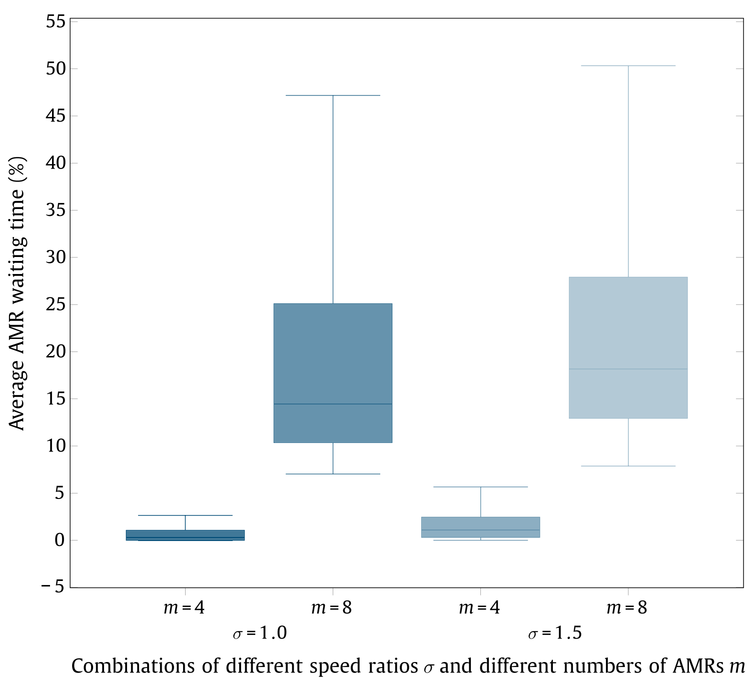

Fig. 10. Average waiting time of the AMRs for the order pickers in percent for different numbers of AMRs and speed ratios.

fleet size have shown that, compared to $m = 4$ AMRs, significantly more customer orders can be processed within their due date assuming $m = 8$ AMRs. Also, we judge the costs associated with the waiting time of AMRs to be less relevant for warehouse managers compared to those of order pickers.

Recall that in the AOPP, the items of a batch cannot be collected by different AMRs. If we were to cancel this assumption and provide a sufficient number of AMRs, it could be ensured that an AMR is already waiting at a handover location when an order picker returns to her handover location after picking, and thus waiting times for order pickers could be reduced. However, we follow this assumption in accordance with the majority of papers from the scientific literature (see, e.g., Henn & Wäscher, 2012) and practical applications. In fact, the consolidation of the items collected by different AMRs into the respective customer order is very time-consuming and error-prone. Moreover, we assume that AMRs travel from left to right along the cross aisle while collecting the batch items. Obviously, this results in longer order picker and AMR waiting times compared to an arbitrary routing of AMRs. Although AMRs are capable to bypass obstacles, we made this assumption to reduce the risk of collision between AMRs. In light of these advantages, we believe that some waiting of order pickers and AMRs is acceptable.

6. Conclusion

In this paper, we investigate a novel warehouse setting, in which human order pickers are supported by AMRs. We present a mathematical model formulation of the resulting AMR-assisted order picking problem (AOPP), which decides on the grouping of customer orders into batches, the assignment of these batches to AMRs, and the sequence according to which these batches are to be processed by the order pickers and the AMRs such that the total tardiness of all customer orders is minimized. To solve the problem, we develop a two-stage heuristic solution approach consisting of an ALNS and an adaption of the well-known NEH heuristic. We use the ALNS component for generating batches, and we implement the NEH-based heuristic to determine the sequence according to which these batches are to be processed.

In numerical studies, we show the positive impact of three algorithmic components on the solution quality, i.e., an SA-based acceptance criterion, destroy operators, and repair operators. We also demonstrate the ability of ALNS/NEH to find high-quality solutions within negligible runtimes on small instances. ALNS/NEH always provides the optimal solution if CPLEX finds an optimum within the given solution time limit. On all instances on which CPLEX is not able to find the optimal solution within the runtime limit, ALNS/NEH finds a solution equal to the upper bound obtained by CPLEX. In addition, we give the following managerial insights:

- The results of our experiments indicate that by adding (or removing) AMRs or by increasing (or decreasing) the AMR speed (e.g., to adapt to different workloads), a large number of customer orders can be completed until the respective due date. The waiting time of AMRs increases for higher AMR speeds and larger fleet sizes, however the costs associated with the waiting of the AMRs are negligible.
- One of the most interesting findings is that (in most cases) a slight increase in the speed ratio leads to a larger reduction of the average total tardiness compared to an expansion of the AMR fleet.
- Speed ratios of $\sigma \geq 2.0$ only contribute to minor additional reductions of the average total tardiness compared to a speed ratio of $\sigma = 1.5$. This finding is definitely interesting for warehouse managers: for safety reasons, speed ratios between $\sigma = 1.0$ and $\sigma = 1.5$ are the norm in real-world warehouses.
- Our experiments indicate that the time spent by order pickers waiting for the next AMR is rather small.

Human order picking in cooperation with AMRs is one of the latest warehousing technologies that is becoming increasingly popular. However, AMR-assistance in traditional picker-to-parts systems has not yet been sufficiently studied. For instance, our AOPP could be extended to address other warehouse layouts (e.g., multi-block parallel-aisle warehouse layouts).

Existing item storage assignment strategies such as turnover-based storage and complementarity-based storage aim at storing frequently requested items in storage locations that are arranged close to the depot. However, in our AMR-assisted picker-to-parts system, also the workload of order pickers, which may vary with different items storage assignment strategies, could be taken into account. Thus, adapting existing or developing new item storage assignment strategies for our AOPP is another interesting field for future research.

References

Azadeh, K., de Koster, R. M. B. M., & Roy, D. (2019). Robotized and automated warehouse systems: Review and recent developments. *Transportation Science, 53*(4), 917–945.

Azadeh, K., Roy, D., & de Koster, R. M. B. M. (2020). Dynamic human-robot collaborative picking strategies. Available at SSRN: https://ssrn.com/abstract=3585396, last accessed on March 9, 2021.

Boysen, N., de Koster, R. M. B. M., & Weidinger, F. (2019a). Warehousing in the e-commerce era: A survey. *European Journal of Operational Research, 277*(2), 396–411.

Boysen, N., Stephan, K., & Weidinger, F. (2019b). Manual order consolidation with put walls: The batched order bin sequencing problem. *EURO Journal on Transportation and Logistics, 8*(2), 169–193.

Chen, T.-L., Cheng, C.-Y., Chen, Y.-Y., & Chan, L.-K. (2015). An efficient hybrid algorithm for integrated order batching, sequencing and routing problem. *International Journal of Production Economics, 159*, 158–167.

Coyle, J. J., Bardi, E. J., & Langley, C. J. (2002). *The management of business logistics. A supply chain perspective* (7th). South-Western/Thomson Learning.

Elsayed, E. A., & Unal, O. I. (1989). Order batching algorithms and travel time estimation for automated storage/retrieval systems. *International Journal of Production Research, 27*(7), 1097–1114.

Frazelle, E. (2020). To zone or not to zone for warehouse order picking? Available online at https://www.linkedin.com/pulse/zone-warehouse-order-picking-dr-ed-frazelle, last accessed on February 8, 2021.

Grosse, E. H., Glock, C. H., & Ballester-Ripoll, R. (2014). A simulated annealing approach for the joint order batching and order picker routing problem with weight restrictions. *International Journal of Operations and Quantitative Management, 20*(2), 65–83.

Henn, S. (2015). Order batching and sequencing for the minimization of the total tardiness in picker-to-part warehouses. *Flexible Services and Manufacturing Journal, 27*(1), 86–114.

Henn, S., & Schmid, V. (2013). Metaheuristics for order batching and sequencing in manual order picking systems. *Computers & Industrial Engineering, 66*(2), 338–351.

Henn, S., & Wäscher, G. (2012). Tabu search heuristics for the order batching problem in manual order picking systems. *European Journal of Operational Research, 222*(3), 484–494.

Hong, S., Johnson, A. L., & Peters, B. A. (2012). Batch picking in narrow-aisle order picking systems with consideration for picker blocking. *European Journal of Operational Research, 221*(3), 557–570.

De Koster, R. M. B. M., Le-Duc, T., & Roodbergen, K. J. (2007). Design and control of warehouse order picking: a literature review. *European Journal of Operational Research, 182*(2), 481–501.

Le-Duc, T., & de Koster, R. M. B. M. (2005). Travel distance estimation and storage zone optimization in a 2-block class-based storage strategy warehouse. *International Journal of Production Research, 43*(17), 3561–3581.

Löffler, M., Boysen, N., & Schneider, M. (2021). Picker routing in AGV-assisted order picking systems. *Forthcoming in: INFORMS Journal on Computing*.

Matusiak, M., de Koster, R. M. B. M., & Saarinen, J. (2017). Utilizing individual picker skills to improve order batching in a warehouse. *European Journal of Operational Research, 263*(3), 888–899.

Napolitano, M. (2012). 2012 Warehouse/DC operations survey: Mixed signals. *Modern Materials Handling, 51*(11), 48–56.

Nawaz, M., Enscore, E. E., & Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega, 11*(1), 91–95.

Palmer, D. S. (1965). Sequencing jobs through a multi-stage process in the minimum total time: A quick method of obtaining a near optimum. *Journal of the Operational Research Society, 16*(1), 101–107.

Petersen, C. G., & Schmenner, R. W. (1999). An evaluation of routing and volume-based storage policies in an order picking operation. *Decision Sciences, 30*(2), 481–501.

Petrow, W. A. (1964). Zusammenstellung des optimalen Kalenderplanes bei der Fertigung der Gruppenfließstraßen. *Fertigungstechnik und Betrieb, 14*(4), 251–254.

Ratliff, H. D., & Rosenthal, A. S. (1983). Order picking in a rectangular warehouse: A solvable case of the traveling salesman problem. *Operations Research, 31*(3), 507–521.

Ropke, S., & Pisinger, D. (2006a). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science, 40*(4), 455–472.

Ropke, S., & Pisinger, D. (2006b). A unified heuristic for a large class of vehicle routing problems with backhauls. *European Journal of Operational Research, 171*(3), 750–775.

Scholz, A., Schubert, D., & Wäscher, G. (2017). Order picking with multiple pickers and due dates – simultaneous solution of order batching, batch assignment and sequencing, and picker routing problems. *European Journal of Operational Research, 263*(2), 461–478.

Shaw, P. (1997). A new local search algorithm providing high-quality solutions to vehicle routing problems. *Technical Report*. Department of Computer Science, University of Strathclyde, Glasgow.

Sokolizin, S. A. (1958). *Gegenstandsabgeschlossene Abschnitte und ihre Rolle bei der Entwicklung der Fließfertigung: Methoden der Fließfertigung im Maschinen- und Gerätebau mit Serienfertigung*. Maschgis, Moskau-Leningrad.

SSI Schäfer (2016). Fahrerloses Transportsystem 2Pick für Behälter, Kartons, Paletten, Stückgutkommissionierung. Available online at https://www.youtube.com/watch?v=1qkT6Nwg9CM, last accessed on March 1, 2020.

Statista (2019). Total retail sales worldwide from 2017 to 2023. Available online at https://www.statista.com/statistics/443522/global-retail-sales, last accessed on March 1, 2020.

Tompkins, J. A., White, J. A., Bozer, Y. A., & Tanchoco, J. M. A. (2010). *Facilities planning* (4th). John Wiley & Sons Inc.

Tsai, C.-Y., Liou, J. J. H., & Huang, T.-M. (2008). Using a multiple-GA method to solve the batch picking problem: Considering travel distance and order due time. *International Journal of Production Research, 46*(22), 6533–6555.

Valle, C. A., Beasley, J. E., & da Cunha, A. S. (2017). Optimally solving the joint order batching and picker routing problem. *European Journal of Operational Research, 262*(3), 817–834.

Žulj, I., Kramer, S., & Schneider, M. (2018). A hybrid of adaptive large neighborhood search and tabu search for the order batching problem. *European Journal of Operational Research, 264*(2), 653–664.

Won, J., & Olafsson, S. (2005). Joint order batching and order picking in warehouse operations. *International Journal of Production Research, 43*(7), 1427–1442.

Wäscher, G. (2004). Order picking: a survey of planning problems and methods. In H. Dyckhoff, R. Lackes, & J. Reese (Eds.), *Supply chain management and reverse logistics* (pp. 323–347). Springer.